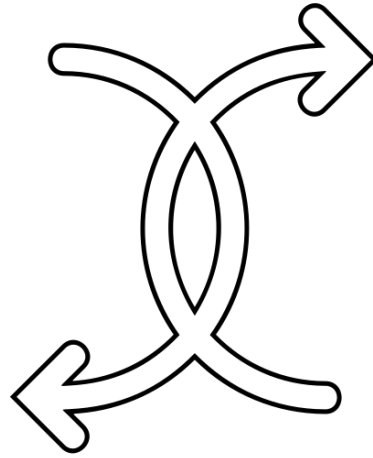




Concorrenza

Eseguire più flussi allo stesso tempo

Lezioni
20-22
Corso 1



Programmazione concorrente

- Un programma concorrente dispone di due o più flussi di esecuzione contemporanei
 - Per perseguire un obiettivo comune
 - Tali flussi possono essere eseguiti in parallelo (se il processore dispone di più core) e/o alternarsi nel tempo, sotto il controllo di uno scheduler
- Ogni processo è **isolato** rispetto agli altri e vive nel suo spazio di indirizzamento (quello che succede nel processo A non ha nessuna interferenza con il processo B)
 - isolamento complica la cooperazione tra processi
 - isolamento imperfetto: possibili interferenze con il file system o con il sistema di rete
- All'atto della creazione, un processo dispone di un **unico flusso di esecuzione**
 - Thread principale
 - Esso può richiedere allo scheduler (con richiesta di intervento del **Sistema Operativo**) la **creazione di altri thread**
 - In C, C++ e Rust il multithreading lo deve scegliere il programmatore facendosi carico delle problematiche
 - Nota: In alcuni linguaggi di programmazione (ad es. Java e JavaScript) il multithreading è implicito.
- Un thread rappresenta una **computazione indipendente**
 - Basata su un proprio stack (pre-allocato all'atto della creazione del thread) collocato nello stesso spazio di indirizzamento in cui operano gli altri thread del processo (**condivisione** dello spazio di indirizzamento)
 - Tale computazione si svolge fino al proprio termine, potendo dare origine ad un risultato o ad un errore

Programmazione concorrente

- Il S.O. e/o le librerie di supporto allocano le risorse fisiche necessarie
 - Lo scheduler ripartisce, nel tempo, l'utilizzo dei core disponibili tra i diversi thread in modo non deterministico
 - Tutti i thread creati sono identificati in modo univoco e viene mantenuto, per quelli in uso, l'indicazione del loro stato di esecuzione
- La gestione dei thread può essere demandata direttamente al S.O.
 - In questo caso si parla di **thread nativi**
- ...oppure gestita da librerie a livello utente, con il supporto parziale del sistema operativo
 - Questi vengono definiti **green thread** o **fibre** e richiedono una certa forma di cooperazione da parte del codice che deve, in modo esplicito, invocare lo scheduler per cedere l'uso della CPU
- C++ e Rust offrono, nella propria libreria standard, supporto per i thread nativi
 - Entrambi offrono librerie di terze parti per il supporto di green thread

Thread nativi

- I diversi sistemi operativi offrono funzionalità simili (ma non identiche) per governare l'interazione di un programma con l'insieme dei thread che lo costituiscono. *L'eterogeneità dei diversi SO ha reso la programmazione concorrente multi-piattaforma un lavoro arduo. Java è stato il primo linguaggio ad offrire un supporto multi-piattaforma alla programmazione concorrente*
- Le funzioni supportate (in tutti i sistemi operativi) sono:
 - **Creazione** di un thread, indicando la funzione che rappresenta la computazione che deve essere svolta e la dimensione dello stack richiesto: questa operazione restituisce un **handle** opaco mediante il quale fare riferimento al thread
 - **Identificazione** del thread corrente, sotto forma di valore univoco a livello di sistema (TID)
 - **Attesa** della terminazione di un thread, a partire dalla sua handle, e accesso al suo stato finale (successo/fallimento): operazione **join**
- Tra le funzioni **non supportate**, spicca la richiesta di **cancellazione** di un thread
 - Questa può solo essere implementata in modo cooperativo dal thread stesso

Cosa implica la concorrenza

- **Riduzione del sovraccarico** dovuto alla comunicazione tra processi
 - Sebbene la suddetta parallelizzazione possa essere fatta anche creando processi separati, il costo di comunicazione e sincronizzazione tra processi è sensibilmente più alto di quello tra thread
 - I thread condividono infatti lo spazio di indirizzamento ed è possibile trasferire la proprietà di strutture dati da un thread ad un altro semplicemente comunicando il puntatore
 - Per ottenere un effetto analogo tra processi differenti, sarebbe necessario serializzare la struttura dati presente nel processo originale, trasferire una copia della rappresentazione ottenuta nel processo destinazione e qui ricostruire una copia della struttura dati

Cosa implica la concorrenza

- Possibilità di **sovrapporre temporalmente** attività di computazione e operazioni di I/O
 - I sistemi operativi tendono ad offrire API bloccanti che arrestano, di fatto, la prosecuzione di un thread fino a che il dato richiesto non è pronto (es.: `read(fd)`, `accept(socket)`, ...)
 - Suddividendo l'algoritmo in più thread, si può cercare sfruttare i tempi di attesa che un dato thread subisce a seguito delle operazioni di I/O per eseguire, in altri thread, operazioni utili al risultato
 - Ciò può permettere un miglioramento delle prestazioni anche in ambiente single core
 - Occorre che la complessità aggiunta dalla suddivisione sia compensata da un effettivo guadagno in termini di tempi di esecuzione

Cosa implica la concorrenza

- Possibilità di sfruttare appieno le capacità di **elaborazione** delle CPU **multicore**
 - Vero parallelismo
 - Più flussi di esecuzione possono svolgersi contemporaneamente, riducendo così il tempo totale di elaborazione
- **Aumento** significativo **della complessità** del programma
 - Nuove fonti e tipologie di errore
 - **Non determinismo dell'esecuzione**
- La memoria **non** può più essere pensata come un **"deposito statico"**
 - I dati scritti al suo interno possono cambiare in conseguenza dell'attività di altri thread
- I thread devono **coordinare l'accesso** alla memoria
 - Tramite opportuni costrutti di sincronizzazione
 - La presenza di cache legate ai singoli core introduce non determinismo nell'ordinamento e nella visibilità delle azioni sulla memoria

Concorrenza in pratica

- Se, all'interno di un processo, sono presenti due o più thread, questi possono procedere indipendentemente nella propria computazione
 - Sebbene sia possibile creare thread che si ignorano reciprocamente e non necessitano alcuno scambio di informazione, sul piano pratico questo avviene molto raramente
- L'utilità di suddividere la computazione globale in più sotto-computazioni nasce, per lo più, dal fatto che ciascuna di esse contribuisce in qualche modo al risultato finale
 - Questo richiede che esista una forma di comunicazione/sincronizzazione tra thread differenti
- I meccanismi soggiacenti alla comunicazione/sincronizzazione interferiscono con le ottimizzazioni usate dai processori per migliorare l'esecuzione
 - Introducendo una serie di **complessità inattese** e lontane dal pensiero comune legato al modello di esecuzione sequenziale

Concorrenza in pratica

- In un sistema single-core, il concetto di thread è puramente una astrazione offerta dal sistema operativo
 - Il processore si limita ad alternare il proprio ciclo di esecuzione basato sulla successione delle micro-operazioni *fetch/decode/execute*, procedendo di istruzione in istruzione secondo la logica del codice macchina
- Il sistema operativo può intervenire in questa sequenza, grazie ad un'interruzione che attiva lo scheduler
 - Questo salva lo stato dei registri in una qualche area di memoria dedicata alle meta-informazioni del thread corrente e li ripristina con il contenuto relativo ad un thread differente (*task switching*)
- Se la CPU è dotata di due o più core, non cambia molto
 - Ciascun core procede indipendentemente dagli altri e lo scheduler provvede a gestire le attività di tutti, allocando di volta in volta i core disponibili ad eseguire l'uno o l'altro thread, secondo le necessità

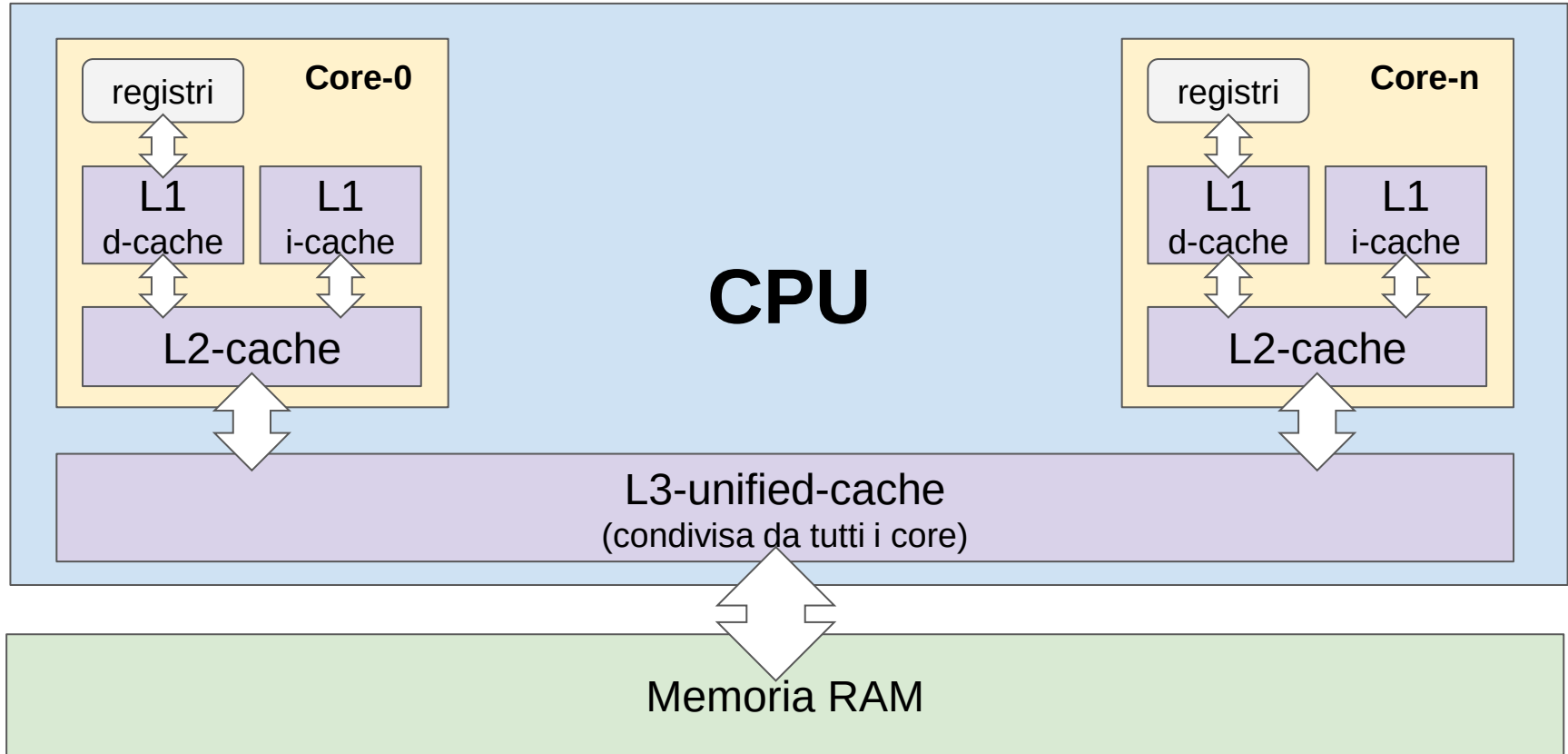
Concorrenza in pratica

- Poiché l'esecuzione di ciascun thread procede indipendentemente da quelle degli altri, un'eventuale necessità di comunicazione deve essere soddisfatta passando per **l'utilizzo di un'area di memoria condivisa**
 - In cui il thread T1 possa depositare le informazioni che intende comunicare al thread T2
- Sebbene i due thread utilizzino lo stesso spazio di indirizzamento e possano, in linea di principio, accedere al dato memorizzato, questa operazione risulta **più complessa** di quanto si possa pensare a prima vista
 - Per rendere la comunicazione utile sul piano pratico, può essere inoltre necessario avvalersi di pattern di interazione che definiscano con precisione i ruoli che le due o più parti coinvolte possono giocare

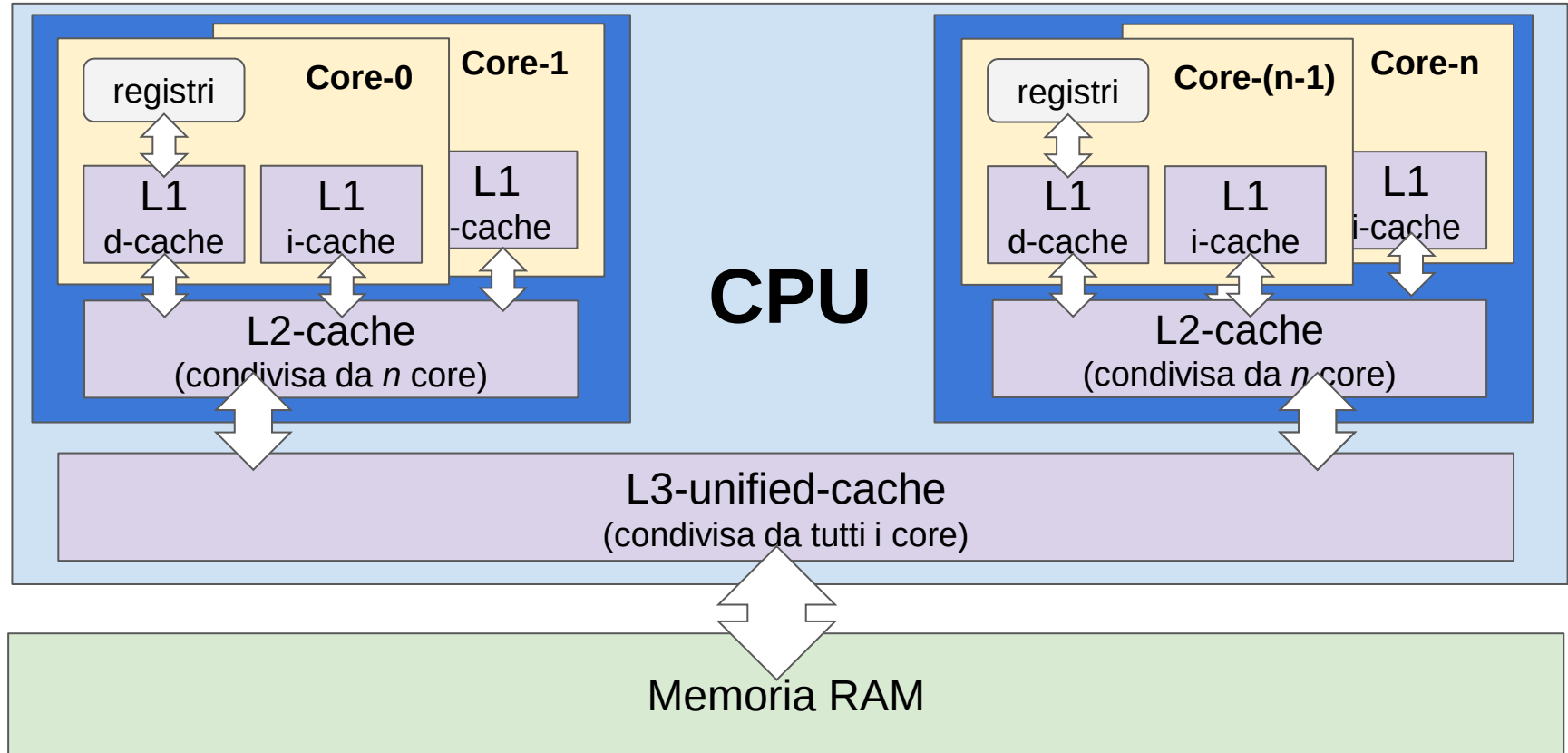
Modello di memoria

- Quando un thread legge il contenuto di una locazione di memoria, può trovare:
 1. Il valore iniziale contenuto nel file eseguibile che è stato mappato in memoria (es: variabile globale inizializzata)
 2. Il valore che **questo stesso thread** ha precedentemente depositato all'interno della locazione
 3. Il valore che è stato depositato da **un altro thread**
 4. un valore **impredicibile** che non è stato scritto da nessuno
- La presenza di cache hardware e il possibile riordinamento delle istruzioni da parte della CPU rendono il terzo caso **problematico** (ossia può succedere che l'operazione di lettura di un thread avviene mentre sta avvenendo l'operazione di scrittura di un altro thread)
 - In generale **non è predicibile** quale valore venga letto senza controllare letture e scritture da parte dei thread
 - Occorre usare un costrutto di sincronizzazione esplicito che permetta di definire l'ordine di esecuzione

Modello di memoria



Modello di memoria (packed cores)



Disabilitare la cache

- Ipotizziamo che un thread abbia modificato una variabile, ma questa variazione non è ancora giunta all'aggiornamento della memoria
- Per evitare che un secondo thread legga da memoria (o da una sua cache) un dato non coerente
- Occorre che le operazioni di scrittura siano fatte attraverso istruzioni Assembly che invalidano la cache
- Istruzioni che rallentano molto l'operazione, ma garantiscono che la lettura sia sicura.

Problemi aperti

- Il problema è principalmente sulle variabili globali e su quelle istanza, meno sulle variabili locali (a meno che il loro indirizzo sia noto ad altri thread)
- **Atomicità**
 - quali istruzioni devono avere effetti indivisibili?
 - si vede che cosa c'era prima o si vede che cosa ci sarà dopo, ma non si vede il durante
 - quali operazioni di memoria hanno effetti indivisibili?
 - Se due thread fanno **accesso alla stessa struttura dati**, rispettivamente in lettura e scrittura non c'è nessuna garanzia su quale delle due operazioni sia **eseguita per prima**
- **Visibilità**
 - Sotto quali condizioni, le scritture compiute da un thread sono visibili da un secondo thread?
 - Se un thread legge un dato che un altro thread sta modificando il valore letto può essere **diverso** sia dal valore **iniziale** che da quello **finale**
 - **Problema legato alla cache**: le scritture che un thread fa devono arrivare fino alla memoria principale per diventare visibili a tutti
- **Ordinamento**
 - Sotto quali condizioni gli effetti di più operazioni effettuate da un thread possono apparire ad altri thread in ordine differente?
 - Problema legato alle ottimizzazioni che il compilatore e il processore può fare che cambiano l'ordine di esecuzione delle istruzioni: può succedere che aggiornamenti diversi abbiano dei tempi di propagazione differenti e dunque la loro visibilità non mantenga lo stesso ordine.

Le risposte dei processori

- Ciascuna famiglia di processori offre una propria risposta ai problemi menzionati

Le risposte dei processori

- La piattaforma **x86** adotta un modello di memoria forte (le operazioni di memoria non vengono ordinate arbitrariamente) **quasi sequenzialmente consistente** ed offre le istruzioni di tipo *fence*, per controllare l'ordine delle operazioni di memoria e forzano il completamento delle operazioni di scrittura, bloccando temporaneamente gli altri core (e invalidando le loro cache) che dovessero cercare di accedere nel frattempo allo stesso segmento di indirizzi:
 - LFENCE (load fence): garantisce che tutte le letture dalla memoria effettuate prima della LFENCE siano completate prima di eseguire letture successive
 - SFENCE (store fence): garantisce che tutte le scritture dalla memoria effettuate prima della SFENCE siano visibili prima di eseguire scritture successive
 - MFENCE (memory fence): garantisce che tutti gli accessi alla memoria iniziati prima della MFENCE siano completati prima di quelle successivi.

Esempio

```
T1
MOV [flag], 1
SFENCE           ; assicura che la scrittura si flag sia visibile
MOV [data], eax

T2
wait: CMP [flag], 1
JNE wait
MFENCE           ; Garantisce che data sia letto dopo il flag
MOV ebx, [data]
```

Le risposte dei processori

- Sulla piattaforma **ARM** viene usato un modello molto più lasco (la CPU può ordinare liberamente load e store), basato su liste di propagazione dei cambiamenti, ed offre le istruzioni di tipo *memory barrier* che consentono l'ordinamento causale delle istruzioni
 - **DMB** (Data Memory Barrier): assicura l'ordine delle operazioni di memoria prima e dopo la barriera, ossia tutte le operazioni di memoria avviate prima della DMB sono completate prima che gli accessi dopo la barriera possano iniziare (si può limitare alle sole letture o alle sole scritture)
 - **DSB** (Data Synchronization Barrier): blocca l'esecuzione finché tutte le operazioni di memoria prima della barriera non siano completate (più forte perché garantisce che nessuna istruzione venga eseguita finché i dati non sono sincronizzati)
 - **ISB** (Instruction Synchronization Barrier): svuota la pipeline delle istruzioni garantendo che tutte le istruzioni dopo la barriera vengano ricaricate dalla memoria.

Le risposte dei processori

- Se tali istruzioni non vengono incluse all'interno del codice generato, **non è garantito un ordinamento predicibile** alle operazioni di **lettura e scrittura di dati condivisi**, in presenza di attività concorrenti
 - C++ e Rust condividono il modello di memoria su cui sono basati e annegano, nelle funzioni di libreria dei tipi dedicati alla concorrenza, tali istruzioni allo scopo di garantire le necessarie proprietà di funzionamento

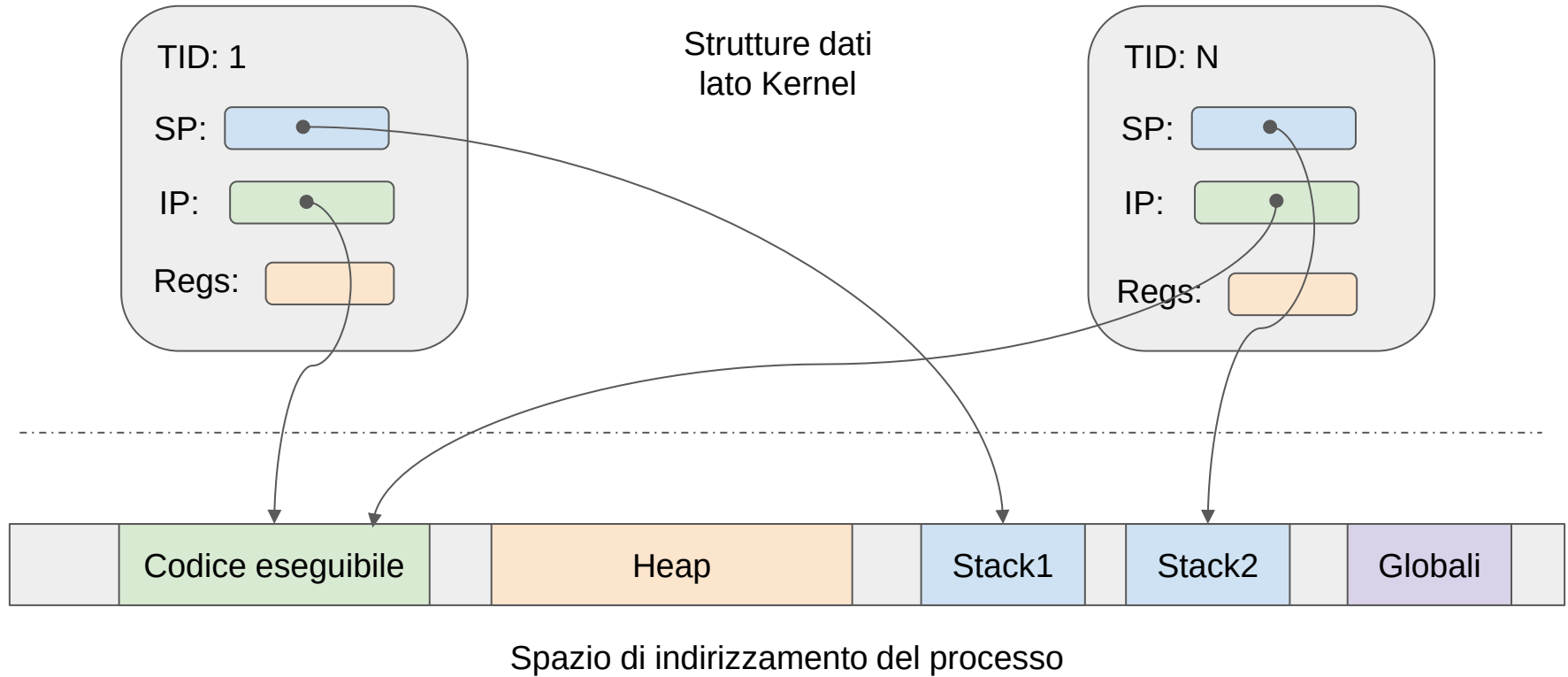
Errori

- L'**uso superficiale** dei costrutti di sincronizzazione porta a:
 - blocchi passivi (che si aspettano reciprocamente => **deadlock**)
 - blocchi attivi (i processi cambiano continuamente il loro stato in funzione dello stato di altri processi, senza fare avanzamenti utili nell'elaborazione, in un ciclo infinito di azioni inutili => **livelock**)
 - assenza di terminazione (non riesco a smettere perché non esiste la possibilità di informarmi che devo smettere => **loop infinito**)
- L'**assenza** di costrutti di sincronizzazione, porta a risultati imprevedibili
 - Anche molto lontani da quanto sarebbe logico aspettarsi
- Si possono verificare **malfunzionamenti casuali**
 - Dovuti al comportamento non deterministico e asincrono dell'esecuzione concorrente
 - Estremamente difficili da riprodurre e da eliminare
- Gli errori possono manifestarsi cambiando la **piattaforma di esecuzione**
 - Oppure soltanto dopo numerose esecuzioni
 - Tipicamente emergono nel momento meno adatto

Thread e memoria

- Il sistema operativo mantiene, al proprio interno, una rappresentazione dei thread presenti all'interno di un dato processo
 - Ad ogni thread viene associato un identificativo univoco (TID - Thread ID), il suo stato di esecuzione (non schedabile, schedabile, in esecuzione sul core i, terminato con errore, terminato senza errore, ...) e le informazioni necessarie a salvare/ripristinare lo stato dei registri interni al processore
 - Provvede inoltre ad allocare, nello spazio di indirizzamento del processo, un blocco di indirizzi contigui destinato ad ospitare lo stack che governerà la sua esecuzione
 - Esiste un numero massimo di thread allocabili
 - Creando un nuovo thread si sottra risorse di memoria per lo stack
- Tutti i thread presenti in un processo **condividono**:
 - Le variabili globali (in Rust, in modalità safe, in sola lettura)
 - Le costanti
 - L'area eseguibile in cui è contenuto il codice
 - Lo heap

Thread e memoria



Esecuzione e non determinismo

- L'esecuzione di ogni singolo thread procede secondo le normali regole sequenziali
 - Per cui è possibile prevedere cosa avvenga "prima" e cosa "dopo"
- Se più thread sono in esecuzione, non è possibile fare assunzioni sulle velocità relative di avanzamento
 - Se non ricorrendo a forme esplicite di **sincronizzazione** e **comunicazione**
- La sincronizzazione può riguardare il raggiungimento di un particolare stato da parte di un thread...
 - Abilitando di conseguenza altri a **procedere**
- ...oppure l'esigenza di un thread di eseguire azioni su aree condivise
 - Allo scopo di **impedire** ad altri di accedere alle stesse aree
- In alcuni casi, all'informazione logica che abilita/impedisce la prosecuzione di altri thread, si accompagna il trasferimento di informazioni più strutturate
 - Che rappresentano l'esito totale o parziale di una computazione avvenuta o la richiesta di elaborazione di ulteriori dati

Esecuzione e non determinismo

```
#include <thread>
#include <iostream>
#include <string>
using namespace std;
void run(string msg) {
    for (int j=0; j<10; j++) {
        cout << msg << to_string(j) << "\n";
        this_thread::sleep_for(chrono::nanoseconds(1));
    }
}
int main() {
    thread t1(run, "aaaa");
    thread t2(run, "bbbb");
    t1.join();
    t2.join();
}
```

C++

```
aaaa0
bbbb0
aaaa1
bbbb1
aaaa2
bbbb2
aaaa3
bbbb3
aaaa4
bbbb4
aaaa5
aaaa6
aaaa7
bbbb5
aaaa8
aaaa9
bbbb6
bbbb7
bbbb8
bbbb9
```

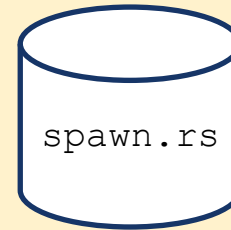

Esecuzione e non determinismo

```
use std::thread;
use std::time::Duration;

fn run(msg: &str) {
    for i in 0..10 {
        println!("{}",msg,i);
        thread::sleep(Duration::from_nanos(1));
    }
}

fn main() {
    let t1 = thread::spawn(|| {run( "aaaa"); });
    let t2 = thread::spawn(|| {run(" bbbb"); });
    t1.join().unwrap();
    t2.join().unwrap();
}
```

Handle del thread



```
aaaa0
aaaa1
aaaa2
bbbb0
bbbb1
bbbb2
aaaa3
aaaa4
aaaa5
aaaa6
aaaa7
bbbb3
bbbb4
bbbb5
aaaa8
aaaa9
bbbb6
bbbb7
bbbb8
bbbb9
```

Esecuzione e non determinismo

- L'output dei programmi precedenti è **solo uno** dei possibili risultati
 - Se lo stesso programma viene eseguito più volte, si ottengono risultati differenti
- È assolutamente possibile (e a volte succede) che tutte le righe di uno dei thread precedano quelle dell'altro
 - In base al numero di core disponibili e alla durata del “quanto” di schedulazione adottato dai sistemi operativi
- L'unica certezza è che le righe che cominciano con "aaaa" sono tra loro ordinate in modo crescente
 - Così come le righe che cominciano con "bbbb"

Esecuzione e non determinismo

- I thread, nel programma precedente, non hanno punti di contatto, ma qualora vi siano punti di contatto (ossia variabili condivise) il **non determinismo** può dare origine a **comportamenti del tutto inattesi**
 - Questo può essere visto facilmente in C/C++, dove l'assenza di restrizioni da parte del borrow checker, non obbliga il programmatore ad avere cura degli aspetti di sincronizzazione
 - Per contro, **Rust si fa garante che un'intera gamma di possibili errori non possano verificarsi**

Esecuzione e non determinismo

```
#include <iostream>
#include <thread>
```

C++

```
int a = 0;          //Questo non è possibile in safe RUST
```

```
void run() {
    while (a >= 0) {
        int before = a;
        a++;
        int after = a;
        if (after-before != 1)
            std::cout << before << " -> " << after
                      << "(" << after-before << ")\n";
    }
}
```

Esecuzione e non determinismo

C++

```
//creo due thread e ne attendo la terminazione

int main() {
    std::thread t1(run);
    std::thread t2(run);

    t1.join();
    t2.join();
}
```

Esecuzione e non determinismo

119944578 -> 119944552 (-26)
123397102 -> 123397584 (482)
128314912 -> 128314956 (44)
395835151 -> 395835236 (85)
396049424 -> 396098482 (49058)
412859791 -> 412859826 (35)
419214490 -> 419214537 (47)
419406880 -> 419406877 (-3)
433982464 -> 433982472 (8)
436005364 -> 436215900 (210536)
441453011 -> 441454010 (999)
446802106 -> 446802106 (0)

Domande

- Esaminando l'uscita del programma precedente, si vedono molti casi in cui la differenza tra **after** e **before** è superiore a **1**
 - In alcuni casi tale valore è anche molto grande: perché?
- Talora capita che la differenza sia **nulla** o **negativa**
 - Come è possibile, se entrambi i flussi incrementano sempre la variabile **a**?



Interferenza

- Si verifica quando più thread fanno accesso a uno stesso dato, **modificandolo**
- La sua presenza dà origine a **malfunzionamenti casuali**, molto difficili da identificare



Istruzione di tipo Read-Modify-Write non atomica

```
#include <iostream>
#include <thread>

int a=0;

void run() {
    while (a!=0) {
        before=a;
        //
        after=a;
        if (after-before!=1)
            std::cout<< before<< " -> " << after<< "("
                << after-before<<")\n";
    }
}
```

Sembra un'azione innocente, ma nasconde due operazioni in cascata:

int temp = a;
a = temp+1;

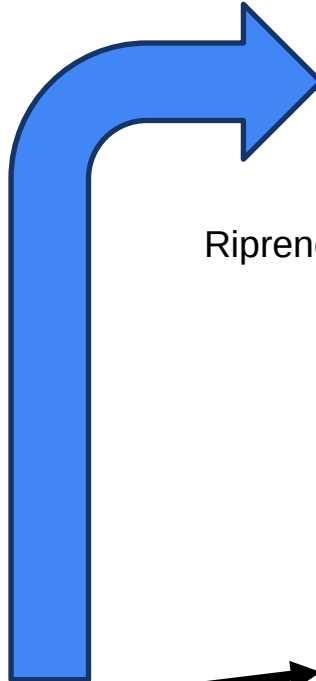
Thread 1

- temp = 1000

Riprende da qua

```
void run() {  
    while (a >= 0) {  
        int before = a;  
        temp = a;  
        a = temp + 1;  
        int after = a;  
        ...  
    }  
}
```

Alla fine del thread 1
a = 1100



Thread 2

- before = 900

Riprende da qua

```
void run() {  
    while (a >= 0) {  
        int before = a;  
        temp = a;  
        a = temp + 1;  
        int after = a;  
        ...  
    }  
}
```

Temp = 1100
After > before + 1

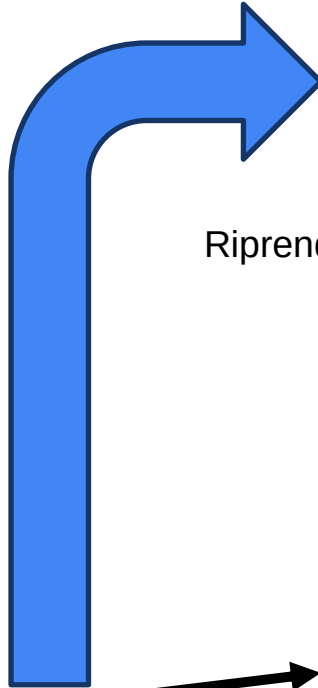
Thread 1

- temp = 900

Riprende da qua

```
void run() {  
    while (a >= 0) {  
        int before = a;  
        temp = a;  
        a = temp + 1;  
        int after = a;  
        ...  
    }  
}
```

Alla fine del thread 1
a = 950



Thread 2

- before = 1000

Riprende da qua

```
void run() {  
    while (a >= 0) {  
        int before = a;  
        temp = a;  
        a = temp + 1;  
        int after = a;  
        ...  
    }  
}
```

Temp = 950
After < before

Sincronizzazione

- Se due thread cercano di accedere in lettura/scrittura ad una stessa variabile si verifica una **corsa critica** (*data race*)
 - In base a condizioni non controllabili dal programmatore (come la presenza di memoria cache, il momento in cui avviene un task switch, le ottimizzazioni fatte dai singoli processori con la predizione della prossima istruzione da eseguire, ...) il dato memorizzato potrebbe essere quello scritto dal primo thread, quello scritto dal secondo oppure un terzo valore **completamente arbitrario**
- L'accesso in lettura/scrittura a variabili il cui contenuto è (potenzialmente) scritto da altri thread è soggetto a diversi vincoli
 - Deve essere preceduto/seguito da istruzioni e proteggano da dati obsoleti presenti nella cache (fence/barrier)
 - Deve avvenire solo quando c'è l'evidenza che il dato non sta venendo modificato da altri
 - Se si sta operando una lettura in attesa di un risultato, si vuole evitare di eseguire cicli continui di polling, che consumano inutilmente cicli di CPU e batteria

Sincronizzazione

- La certezza che l'operazione di accesso corrisponde alla variabile in memoria è garantito da **apposite istruzioni macchina** (*fence/barrier*)
 - Che dipendono dal processore responsabile dell'esecuzione del codice
- La certezza che l'accesso ai dati sia effettuato solo da un thread alla volta è garantito da **invarianti a livello sistema** che può essere fornita solo da meccanismi offerti dal sistema operativo (*mutex* e *condition variable*) che, controllando la schedulazione dei thread, può farsi carico che avvenga / non avvenga una determinata condizione
- Ne consegue che i meccanismi di sincronizzazione dipendono dalla coppia processore/sistema operativo, che collettivamente definiscono l'interfaccia binaria dell'applicazione (ABI - Application Binary Interface)
- Le librerie standard dei diversi linguaggi di programmazione si fanno (talora) carico di standardizzare tale comportamento, offrendo API comuni a livello di codice sorgente
 - Come nei casi di C++11 e successivi e di Rust

Sincronizzazione

- La certezza che l'operazione di accesso a dati condivisi sia **apposite istruzioni macchina** (*fence/flush*)
 - Che dipendono dal processore responsabile
- La certezza che l'accesso ai dati sia **invarianti a livello sistema** che può essere fornita solo da meccanismi offerti dal sistema operativo (*mutex* e *condition variable*) che, controllando la schedulazione dei thread, può farsi carico che avvenga / non avvenga una determinata condizione
- Ne consegue che i meccanismi di sincronizzazione dipendono dalla coppia processore/sistema operativo, che collettivamente definiscono l'interfaccia binaria dell'applicazione
 - ABI - Application Binary Interface
- Le librerie standard dei diversi linguaggi di programmazione si fanno (talora) carico di standardizzare tale comportamento, offrendo API comuni a livello di codice sorgente
 - Come nei casi di C++11 e successivi e di Rust

Mutex genera la barriera e garantisce che entrando nel blocco di codice venga invalidata la linea di cache corrispondente alla variabile condivisa e quando esco da quel blocco di codice venga fatta la flush della cache

Sincronizzazione

- La certezza che l'operazione di accesso corrisponda a **apposite istruzioni macchina** (*fence/barrier*)
 - Che dipendono dal processore responsabile dell'esecuzione
- La certezza che l'accesso ai dati sia effettuato in modo **invarianti a livello sistema** che può essere fornita solo da meccanismi offerti dal sistema operativo (*mutex* e *condition variable*) che, controllando la schedulazione dei thread, può farsi carico che avvenga / non avvenga una determinata condizione
- Ne consegue che i meccanismi di sincronizzazione dipendono dalla coppia processore/sistema operativo, che collettivamente definiscono l'interfaccia binaria dell'applicazione
 - ABI - Application Binary Interface
- Le librerie standard dei diversi linguaggi di programmazione si fanno (talora) carico di standardizzare tale comportamento, offrendo API comuni a livello di codice sorgente
 - Come nei casi di C++11 e successivi e di Rust

Condition Variable fa attendere un thread fino a quando una condizione non si verifica senza consumare CPU e, appoggiandosi ad un mutex garantiscono una barriera di protezione e coerenza della memoria.

Strutture native di sincronizzazione



Windows

- Strutture dati utente
 - **CriticalSection**
 - **SRWLock**
 - **ConditionVariable**
- Oggetti kernel
 - **Mutex**
 - **Event**
 - **Semaphore**
 - **Pipe**
 - **Mailslot**
 - ...



Linux

- Strutture dati utente
 - **pthread_mutex**
 - **pthread_cond**
- Oggetti kernel
 - **Semaphore**
 - **Pipe**
 - **Signal**
 - **Futex**



Correttezza

- Tutti gli oggetti **condivisi mutabili** devono godere di questa proprietà
 - Occorre fare in modo che **non capiti mai** che **un thread** "operi" su un dato, alterandone il contenuto mentre **un altro** sta già operando sullo stesso oggetto
 - non devono essere visibili **stati transitori** dell'oggetto, dovuti al meccanismo di aggiornamento in cui solo una parte dell'informazione contenuta è cambiata (come per la proprietà di isolamento nelle basi di dati).



Correttezza

- In quasi tutti i linguaggi di programmazione, è compito del programmatore riconoscere **quando**, **dove** e **come** utilizzare la sincronizzazione
 - Un uso sbagliato porta a **risultati disastrosi**
- Occorre dimostrare la **correttezza formale** dell'algoritmo complessivo
 - Garantendo che, qualunque sia l'ordine di esecuzione scelto dal sistema operativo o la latenza introdotta dai sottosistemi di memoria, il programma resta corretto
 - Questo è possibile grazie alla presenza di primitive offerte dal sistema operativo / compilatore / librerie di esecuzione che garantiscono alcune **invarianti di basso livello**
- Se un programma non è corretto, è inutile pensare di testarlo o ottimizzarlo
 - L'esito dei test è non deterministico: potrebbero passare per pura combinazione fortuita (o anche probabile) delle condizioni al contorno, ma non danno alcuna garanzia
 - Le ottimizzazioni non farebbero che esacerbare il problema, rendendone più difficile la comprensione



Correttezza

- In Rust, le limitazioni imposte dal borrow checker sulla esclusività dell'accesso in scrittura (unico possessore e unico riferimento in scrittura), unite all'utilizzo di tratti che modellano il comportamento che un tipo esibisce quando viene passato da un thread ad un altro, diventano **garanti della correttezza degli accessi alle variabili condivise** a livello di compilazione
 - Trasformando errori in esecuzione difficili da identificare e replicare in errori di compilazione
 - Questo ha portato a definire questo aspetto di Rust come **fearless concurrency**
- Resta comunque responsabilità del programmatore la comprensione dell'algoritmo e la garanzia di terminazione dello stesso, così come la verifica dell'assenza di blocchi attivi e passivi che possono impedire il procedere dell'algoritmo

Accesso condiviso: le possibili soluzioni

- **Tipi Atomic**

- Alla base di tutti i meccanismi di accesso condiviso ci sono istruzioni apposite, offerte dai singoli processori, volte a garantire operazioni di tipo Read-Modify-Write di tipo atomico (cioè, non interrompibili e non osservabili nei loro stati intermedi), su CPU single- e multi-core
- Tali operazioni sono limitate a tipi semplici (booleani, interi, puntatori) e sono esposte dalle librerie standard di C++ e Rust attraverso opportune astrazioni, che incapsulano l'utilizzo di barriere di memoria

- **Mutex**

- Per estendere le garanzie di atomicità e dipendenza causale a strutture dati più complesse, occorre introdurre il concetto di Mutex
- Essi estendono il principio della mutua esclusione a thread differenti
- Un mutex può essere libero o posseduto da un singolo thread
- Se un secondo thread cerca di ottenere il possesso del mutex mentre è in uso da parte di un altro thread, rimane in attesa (senza consumare cicli di CPU) fino a che esso non viene rilasciato

- **Condition variable**

- In alcuni casi, occorre attendere - senza consumare cicli di CPU - che si verifichi una condizione più complessa del semplice rilascio di un mutex da parte di un thread
- Una condition variable permette di realizzare tale attesa, a condizione che il thread che causa l'avverarsi della condizione si occupi di segnalarlo, generando una notifica tramite appositi metodi
- Una condition variable può essere usata solo in coppia con un mutex

Uso dei thread

- Le API dei S.O. permettono la gestione del ciclo di vita dei thread
 - Creazione e terminazione di thread
 - Meccanismi di sincronizzazione
 - Aree private di memoria
- I dettagli relativi a ciascuna piattaforma differiscono alquanto
 - Rendendo complessa la portabilità delle applicazioni
- La versione 2011 del linguaggio C++ ha introdotto una standardizzazione nella creazione e gestione dei thread
 - Tale standardizzazione, tuttavia, nello sforzo di uniformare i comportamenti, nasconde le peculiarità offerte dai singoli sistemi operativi per gestire i casi particolari connessi alla computazione, come cancellazione e fallimento
- Una standardizzazione analoga è offerta dalla libreria standard di Rust
 - Con una maggiore attenzione alla gestione del fallimento

Thread in C++

- La classe `std::thread` offre il supporto per la creazione di un thread nativo e la gestione del suo ciclo di vita
 - Il costruttore di tale classe accetta come parametro un oggetto callable (puntatore a funzione, oggetto funzionale, funzione lambda) e eventuali ulteriori parametri da passare a tale oggetto
 - Quando il costruttore ritorna, è presente il thread nativo creato al suo interno si trova nello stato *runnable*, e può essere considerato schedulabile a tutti gli effetti
- L'oggetto thread inizializzato mantiene il riferimento opaco (handle) al thread nativo
 - Si può attendere la terminazione del thread nativo invocando il metodo bloccante `join()`
 - Oppure si può disgiungere l'oggetto dal thread nativo, invocando il metodo `detach()`
- Il distruttore dell'oggetto verifica, quando viene invocato, che il thread sia effettivamente terminato o sia stato distaccato
 - Se nessuna delle due opzioni è verificata, l'intero processo viene arrestato invocando la funzione `std::terminate()`
- Se la computazione svolta dal thread genera un'eccezione non gestita all'interno del thread stesso, l'intero processo viene terminato
 - Tramite la funzione `std::terminate()`

Thread in Rust



- In Rust si crea un thread nativo attraverso la funzione `std::thread::spawn(...)`
 - Essa accetta una closure (priva di parametri) che rappresenta la computazione che il thread deve svolgere
 - Ritorna una struct di tipo `std::thread::JoinHandle<T>`, dove `T` rappresenta il tipo restituito dalla computazione del thread (ovvero il tipo ritornato dalla funzione lambda)
- Per sapere quando la computazione del thread è terminata e quale valore abbia prodotto, occorre utilizzare il metodo `join()` offerto dalla handle
 - Tale metodo restituisce un'enumerazione di tipo `std::thread::Result` che contiene, nell'opzione `Ok`, il valore finale e nell'opzione `Err` il valore eventualmente passato alla macro `panic!`, nel caso in cui sia stata invocata nel corso della computazione del thread stesso
- Non si crea nessun rapporto di parentela tra thread creatore (quello in cui si invoca `spawn(...)`) e thread creato
 - Né occorre che l'uno sopravviva all'altro
 - Il thread creato non sa neanche chi è il thread creatore
 - Quando l'handle di un thread esce dello scope e viene rilasciata, non c'è più modo di avere notizie (dirette) sull'esito del thread creato, che acquisisce lo stato `detached`
- La standard library di Rust assegna ad ogni thread un identificatore accessibile attraverso la chiamata `thread::current().id()`

```
use std::thread;
```

```
fn main() {  
    let t1 = thread::spawn(f);  
    let t2 = thread::spawn(f);  
  
    println!("Hello from the main thread.");  
    t1.join().unwrap();  
    t2.join().unwrap();  
    println!("End.");  
}  
  
fn f() {  
    println!("Hello from another thread!");  
  
    let id = thread::current().id();  
    println!("This is my thread ID: {id:?}");  
}
```



thread.rs

Output Locking: la macro `println!` utilizza la funzione `std::io::Stdout::Lock()` per garantire che la stringa stampata non sia interrotta.

```
use std::thread;
```

```
fn main() {
```

```
    let data = vec![1, 2, 3, 4];
```

```
    // provare con
```

```
    // let data = vec![];
```

```
    // e vedere che cosa succede
```

```
    // Creiamo un nuovo thread e trasferiamo il possesso del vettore "data"
```

```
    let handle = thread::spawn(move || {
```

```
        let len:usize= data.len();
```

```
        let somma: usize = data.iter().sum();
```

```
        somma/len
```

```
    });
```

```
    println!("Il thread principale non ha accesso ai dati !");
```

```
    // println!("Ecco un vettore: {:?}", data);
```

```
    println!("Il thread principale aspetta");
```

```
    let average = handle.join();
```

```
    match average {
```

```
        Ok(res) => {println!("La media è {:?}", res);},
```

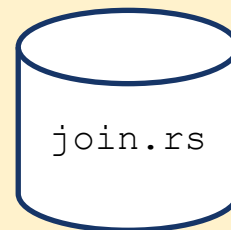
```
        Err(err) => {println!("Errore {:?}", err);}
```

```
    }
```

```
    println!("Il thread principale termina.");
```

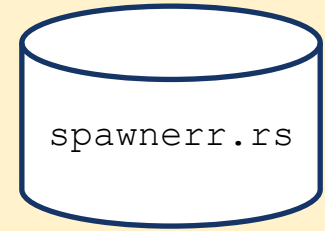
```
}
```

Cattura per movimento delle
variabili che diventano di
proprietà del thread




```
use std::thread;
```

```
fn main() {  
    let mut numero = 5;  
  
    let handle = thread::spawn( || {  
        let x = 2;  
        numero += x;  
        println!("Numero incrementato di {}. Nuovo valore: {}", x, numero);  
        numero  
    });  
  
    let result = handle.join();  
    match result {  
        Ok(res) => {println!("Il risultato è {:?}", res);},  
        Err(err) => {println!("Errore {:?}", err);}  
    }  
}
```



Questo programma non compila? Come mai?

Rispondete sul canale Discord Rust

<https://discord.gg/Y7kjsx8T2>

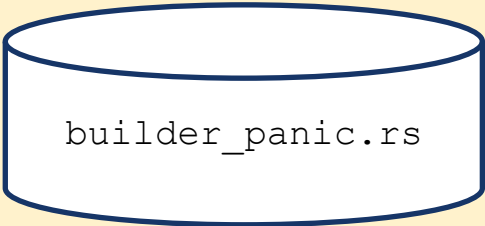
Configurare un thread

- E' possibile configurare un thread prima di lanciare la sua esecuzione tramite la struct **std::thread::Builder**
 - Permette di assegnare al thread un nome a scelta e di definire la dimensione dello stack da associare al thread
 - Il metodo **spawn(...)** consuma l'oggetto Builder, crea il thread corrispondente e restituisce un enum di tipo **io::Result<JoinHandle>**

```
fn main() {  
    use std::thread;  
  
    let mut a = vec![1, 2, 3];  
  
    let builder = thread::Builder::new()  
        .name("t1".into())  
        .stack_size(100_000);  
    let handler = builder  
        .spawn( move || {  
            a.push(4);  
            let x: i32 = a.iter().sum();  
            println!("x: {}", x); // Stampa: x: 10  
        });  
    handler.unwrap().join().unwrap();  
    println!("Fine");  
}
```



```
fn main() {  
    use std::thread;  
  
    let a = vec![1, 2, 3];  
  
    let builder = thread::Builder::new()  
        .name("t1".into())  
        .stack_size(100_000);  
  
    let handler = builder  
        .spawn( move || {  
            println!("{}", a[4]); // panica. Viene fornito il nome del thread.  
        });  
  
    let result = handler.unwrap().join();  
    match result {  
        Ok(()) => {println!("Terminato Thread");},  
        Err(err) => {println!("Errore {:?}", err);}  
    }  
    println!("Fine");  
}
```



builder_panic.rs

thread 't1' panicked at src\main.rs:12:29:

Che cosa posso *passare* ad un thread ?

- Un thread può accedere a dati che vengono dal contesto
- Rust limita i dati che possono essere trasferiti introducendo 2 tratti marker (ossia privi di metodi).

I tratti della concorrenza: marcatori di sicurezza

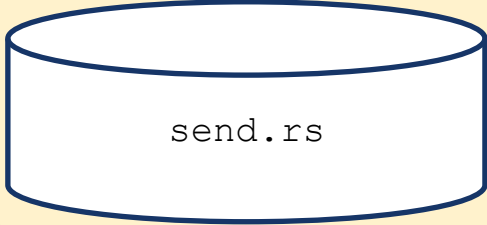


- Nell'ambito del proprio sforzo di garantire la correttezza degli accessi alla memoria e l'assenza di comportamenti non definiti, Rust introduce due tratti marcatori (senza metodi), il cui scopo è fornire indicazioni sul comportamento di un tipo in un contesto multi-thread
 - Il tratto `std::marker::Send` è applicato automaticamente a tutti i tipi che possono essere trasferiti in sicurezza da un thread ad un altro, ovvero in grado di garantire che non è possibile avere accessi al loro contenuto **contemporaneamente** (un tipo che gode del tratto Send può essere ceduto ad un thread)
 - Il tratto `std::marker::Sync` è applicato automaticamente a tutti i tipi `T` tali che `&T` risulta avere il tratto `Send`, ovvero che **possono essere condivisi** in sicurezza tra thread differenti, senza creare problemi di comportamenti non definiti (la reference in lettura ad un tipo che gode del tratto Send può essere passata ad un thread)
- **Riferimenti (mutabili e immutabili), RefCell e puntatori nativi non hanno** il tratto `Send`
 - L'esecuzione indipendente dei thread non consente infatti al borrow checker di fornire le proprie garanzie di correttezza
- **Rc non ha** il tratto `Send`
 - Implica operazioni di aggiornamento che causano race condition.

I tratti della concorrenza

- I tipi elementari godono del tratto Send
- I tipi composti (struct, tuple, enum, array) godono del tratto **Send** se tutti i loro campi lo posseggono
 - Se un tipo dispone del tratto **Send**, è lecito passarlo **per valore** ad altri thread
 - L'uso del movimento (o della copia, là dove possibile) garantisce la non contemporaneità degli accessi
- **Rc** e **RefCell** (non godono del tratto Send): non possono essere usati in contesto multi-thread.
 - Se un tipo dispone del tratto **Sync**, è lecito passarlo **come riferimento non mutabile** ad altri thread o, in altre parole, un tipo è **Sync** se è lecito accedervi in modo concorrente a partire da un riferimento non mutabile
 - I tipi che implementano una mutabilità interna (come **Cell**) non dispongono del tratto **Sync**.

```
fn main() {  
    // Creiamo una variabile condivisa tra più thread.  
    let shared_value = 42;  
  
    // Creiamo due thread che accedono alla variabile condivisa.  
    let handle1 = std::thread::spawn(move || {  
        println!("Thread 1: Valore condiviso = {}", shared_value);  
        let a = shared_value + 1;  
        println!("Thread 1: Valore calcolato = {}", a);  
    });  
  
    let handle2 = std::thread::spawn(move || {  
        println!("Thread 2: Valore condiviso = {}", shared_value);  
        let b = shared_value + 10;  
        println!("Thread 2: Valore calcolato = {}", b);  
    });  
  
    handle1.join().unwrap();  
    handle2.join().unwrap();  
}
```



send.rs

```
struct Message {  
    content: String,  
    sender_id: u32,  
}  
  
fn main() {  
    let msg = Message {  
        content: "Ciao dal thread principale!".to_string(),  
        sender_id: 42,  
    };  
  
    // Crea un nuovo thread e invia il messaggio.  
    std::thread::spawn(move || {  
        println!("Messaggio ricevuto: {}", msg.content);  
    })  
        .join()  
        .unwrap();  
}
```




```
use std::cell::Cell;
use std::thread;

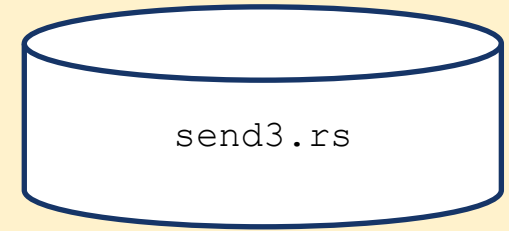
fn main() {
    // Crea un Cell contenente un intero
    let my_cell = Cell::new(42);

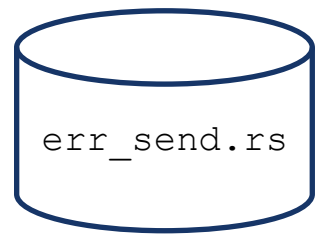
    // Clona il Cell per entrambi i thread
    let thread1_cell = my_cell.clone();
    let thread2_cell = my_cell.clone();

    // Thread 1: Incrementa il valore nel Cell
    let handle1 = thread::spawn(move || {
        thread1_cell.set(thread1_cell.get() + 1);
        println!("Thread1: Valore nel Cell: {}", thread1_cell.get());
    });

    // Thread 2: Legge il valore dal Cell
    let handle2 = thread::spawn(move || {
        println!("Thread2: Valore nel Cell: {}", thread2_cell.get()-1);
    });

    // Attendi che entrambi i thread terminino
    handle1.join().unwrap();
    handle2.join().unwrap();
    println!("Main Thread: Valore nel Cell: {}", my_cell.get());
}
```





- E' possibile creare thread solo se i dati catturati dalla funzione lambda che ne descrive la computazione e il suo tipo di ritorno hanno il tratto **Send**
 - In mancanza di questa garanzia, il borrow checker genera un errore di compilazione, rilevando la propria impossibilità a garantire la correttezza di quanto si sta chiedendo di eseguire

```
use std::thread;
use std::rc::Rc;
fn main() {
    let data1 = Rc::new(1);
    let data2 = Rc::clone(&data1);
    println!("t0: {}", *data1);

    let jh = thread::spawn(move || {
        println!("t1: {}", *data2);
    });
    jh.join().unwrap();
}
```



```
error[E0277]: `Rc<i32>` cannot be sent
between threads safely
  --> src/main.rs:7:12
       |
7      |         let jh = spawn(move || {
       |         _____^
       |         |           `Rc<i32>` cannot be
       |         | sent between threads safely
```

Soluzione



sendarc.rs

```
use std::thread;
use std::sync::Arc;

fn main() {
    let data1 = Arc::new(1);
    let data2 = Arc::clone(&data1);
    println!("t0: {}", data1);

    let jh = thread::spawn(move || {
        println!("t1: {}", data2);
    });

    jh.join().unwrap();
}
```

Attenzione!!!

- Rust non lascia spostare un riferimento con durata limitata (non 'static) in un thread perché non può garantirne la validità nel thread.

```
use std::thread;
```



```
fn main() {  
    let string = "Hello, world!".to_string();  
    thread::spawn( || {  
        println!("Stringa: {}", &string);  
    }).join().unwrap();  
}
```

syncerr1.rs

```
use std::thread;
```



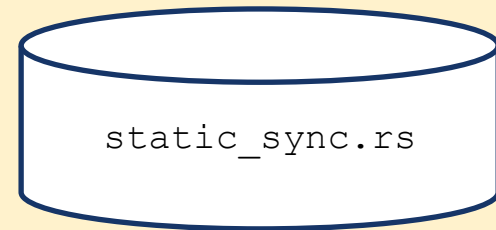
```
fn main() {  
    let string = "Hello, world!";  
    thread::spawn( || {  
        println!("Stringa: {}", &string);  
    }).join().unwrap();  
}
```

syncerr2.rs

```
use std::thread;
static STRING: &str = "Viva Rust";

fn main() {

    thread::spawn( || {
        println!("Stringa: {}", STRING);
    }).join().unwrap();
    println!("Stringa: {}", STRING);
}
```



Sync ??

- Allora come capperò facciamo a passare un riferimento ad un thread, per dati che godono del tratto Sync?
- Lo vedremo tra poco.

Thread principale

- Quando il thread principale termina, fa terminare tutti gli eventuali thread ancora attivi

```
fn main() {  
    let vettore = vec![1, 2, 3, 4, 5];  
    let vettore2 = vettore.clone();  
    let vettore3 = vettore.clone();  
  
    std::thread::spawn(move || {  
        thread::sleep(Duration::from_secs(1)); // provatelo togliendo i tempi di attesa  
        println!("Thread 1: {:?}", vettore2);  
    });  
  
    std::thread::spawn(move || {  
        thread::sleep(Duration::from_secs(1)); // provatelo togliendo i tempi di attesa  
        println!("Thread 2: {:?}", vettore3);  
    });  
  
    println!("Fine del programma, Vettore: {:?}", vettore);  
}
```



mainthread.rs

Fine del programma, Vettore: [1, 2, 3, 4, 5]

Scoped Thread

- Se un thread viene creato mediante la primitiva `std::thread::spawn(...)`, il compilatore non può fare assunzioni sulla sua durata
 - Di conseguenza, impedisce l'utilizzo di riferimenti condivisi tra la funzione lambda del thread e la funzione all'interno della quale il thread viene creato
- La libreria standard offre un ulteriore modo di creare un thread, tramite la funzione `std::thread::scope(|s: std::thread::Scope| { ... })`
 - Essa accetta come parametro una funzione lambda il cui compito è racchiudere l'intero ciclo di vita dei thread creati al suo interno
 - Il parametro `s` passato a tale funzione offre il metodo `.spawn(...)` mediante il quale è possibile creare nuovi thread
- Terminata l'esecuzione della funzione lambda, la funzione `scope(...)` non ritorna fino a che tutti i thread creati al suo interno non sono terminati
 - Questo permette al borrow checker di considerare corretto l'uso di riferimenti a variabili locali, dato che la loro durata sarà almeno pari a quella della funzione `scope(...)` e, di conseguenza, a quella dei thread creati al suo interno


```
use std::thread;
```

```
fn main() {  
    let mut numbers = vec![1, 2, 3];
```

```
    thread::scope(|s| {  
        s.spawn(|| {  
            println!("length: {}", numbers.len());  
            // riferimento a variabile che non deve essere catturata con move  
        });
```

Passo il riferimento
perché Vec<i32> gode
del tratto Sync

scope.rs

```
        s.spawn(|| {  
            for n in &numbers {  
                println!("{}", n);  
            }  
        });  
    });
```

Passo il riferimento
perché Vec<i32> gode
del tratto Sync

```
    // Solo quando entrambi i thread saranno terminati si proseguirà  
    numbers.push(4); // non ci sono più riferimenti, si può modificare
```

```
    println!("{:?}", numbers);
```

```
}
```

Modelli di concorrenza

- La libreria standard di Rust supporta due modelli base per la realizzazione di programmi concorrenti
 1. La condivisione di dati basata su sincronizzazione degli accessi ad una **struttura dati condivisa**, a cui tutti i thread interessati possono accedere in lettura e scrittura
 2. La condivisione di dati basata sullo **scambio di messaggi** che prevede uno o più mittenti ed un solo destinatario
- Sono inoltre disponibili librerie esterne che supportano ulteriori modelli
 - La libreria **actix** supporta il modello degli attori
 - La libreria **rayon** supporta il modello work stealing
 - La libreria **crossbeam** permette la condivisione di dati memorizzati nello stack del thread genitore con i thread figli

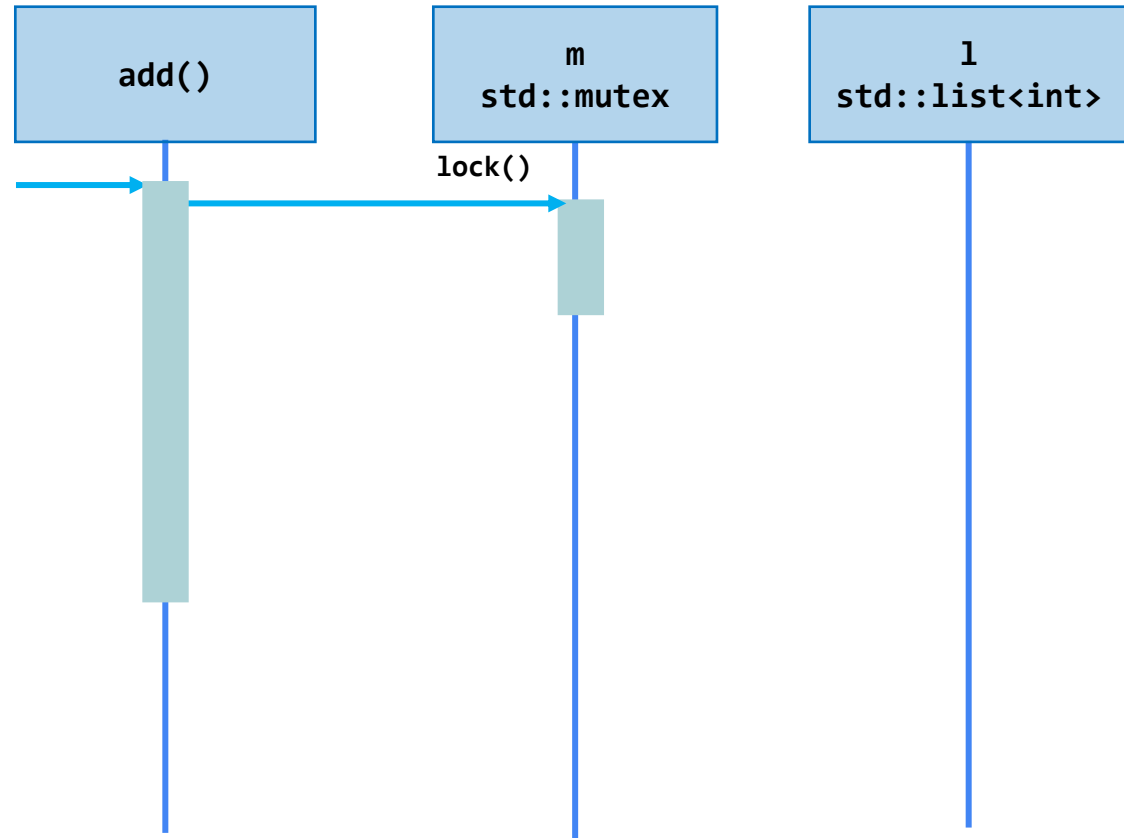
Condivisione dello stato

- Per poter condividere dati modificabili tra thread differenti, occorre disporre di un qualche **meccanismo che permetta, ad un solo thread alla volta, di acquisire il permesso di modifica**
 - Bloccando lo svolgimento degli altri thread che dovessero richiedere accesso alla stessa risorsa
- Il modo più semplice di ottenere questo comportamento è attraverso l'uso di un **mutex** (MUTual EXclusion lock)
 - Costrutti di questo tipo sono offerti nativamente dai sistemi operativi e sono riesportati in modo indipendente dalla piattaforma dalle librerie standard C++ e Rust
- Gli oggetti nativi offerti dai sistemi operativi offrono due metodi: **lock()** e **unlock()**
 - Invocando lock(), un thread richiede il possesso del mutex: se questo non può essere garantito al momento, perché il mutex è in uso ad un altro thread, **l'invocazione si blocca** fino a che il mutex non è stato rilasciato dall'attuale possessore
 - E' lecito invocare unlock() solo se il thread che lo esegue è l'attuale possessore del mutex

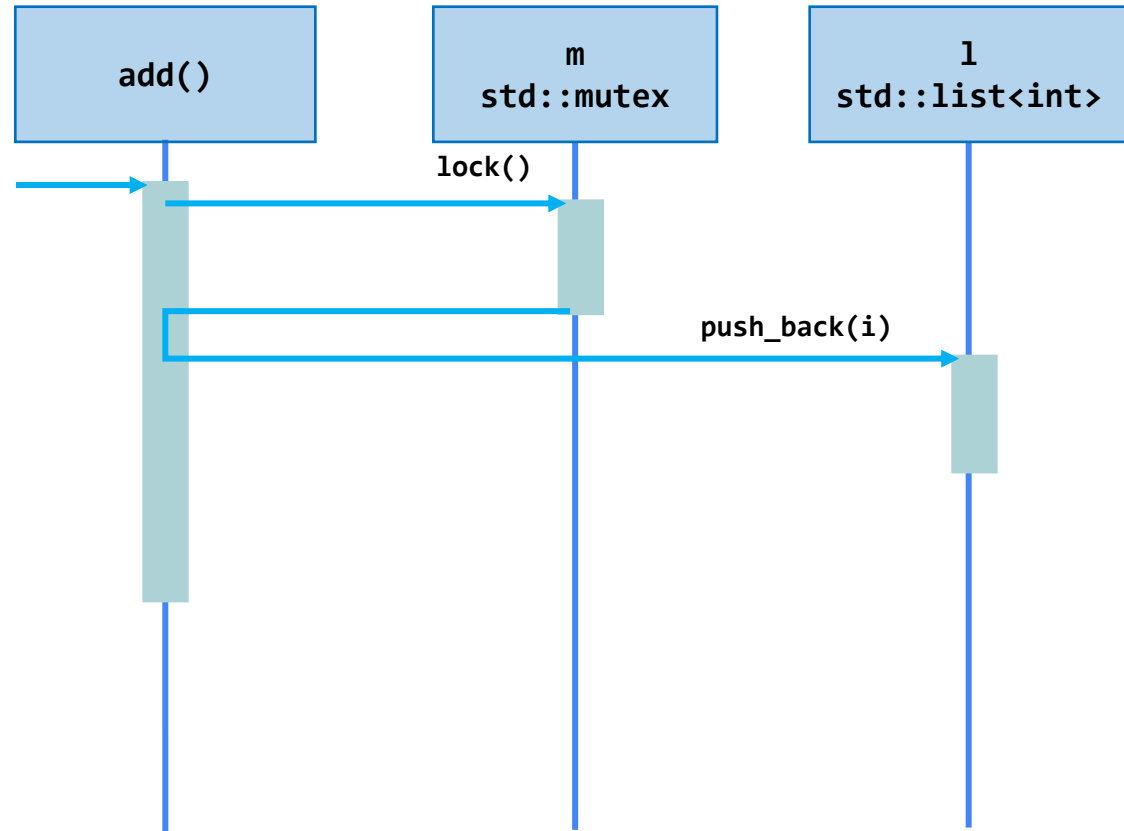
Condivisione dello stato

- Entrambi i metodi **lock()** e **unlock()** includono una barriera di memoria
 - Essa garantisce la visibilità delle operazioni eseguite fino a quel punto dagli altri thread
 - Permette di imporre la dipendenza causale
- Sul piano pratico, occorre associare ad ogni risorsa condivisa un mutex
 - Che deve essere **sempre** acquisito prima di fare accesso (sia in lettura che in scrittura) alla risorsa
- Nelle astrazioni base offerte dai sistemi operativi e nell'implementazione offerta in C++, **non c'è una corrispondenza sintattica tra un mutex e la struttura dati che questo protegge**
 - La relazione, in questi ambienti, è nella mente (e nelle intenzioni) del programmatore
- Un mutex può, in linea di principio, proteggere molte strutture diverse
 - Ma riduce il grado di parallelismo complessivo del programma

```
std::list<int> l;  
std::mutex m;  
  
void add(int i){  
    m.lock();  
    l.push_back(i);  
    m.unlock();  
}
```



```
std::list<int> l;  
std::mutex m;  
  
void add(int i){  
    m.lock();  
    l.push_back(i);  
    m.unlock();  
}
```



```

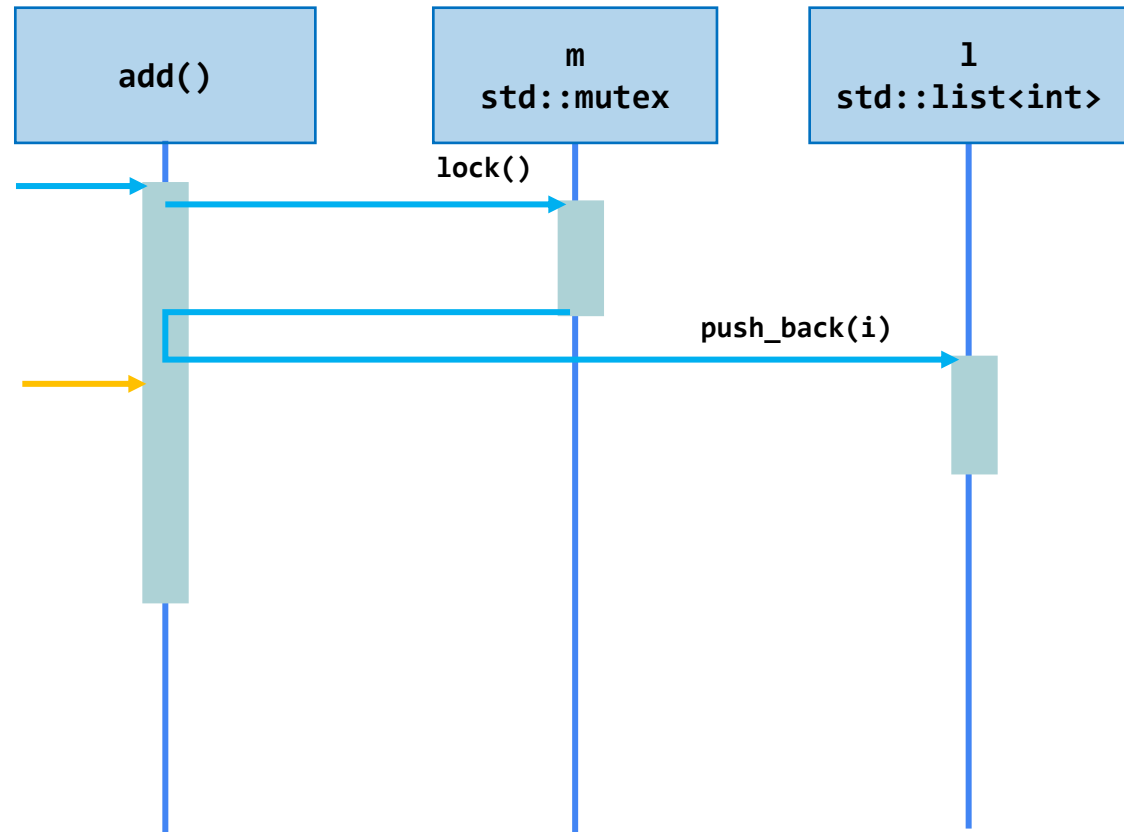
std::list<int> l;
std::mutex m;

void add(int i){
    m.lock();

    l.push_back(i);

    m.unlock();
}

```

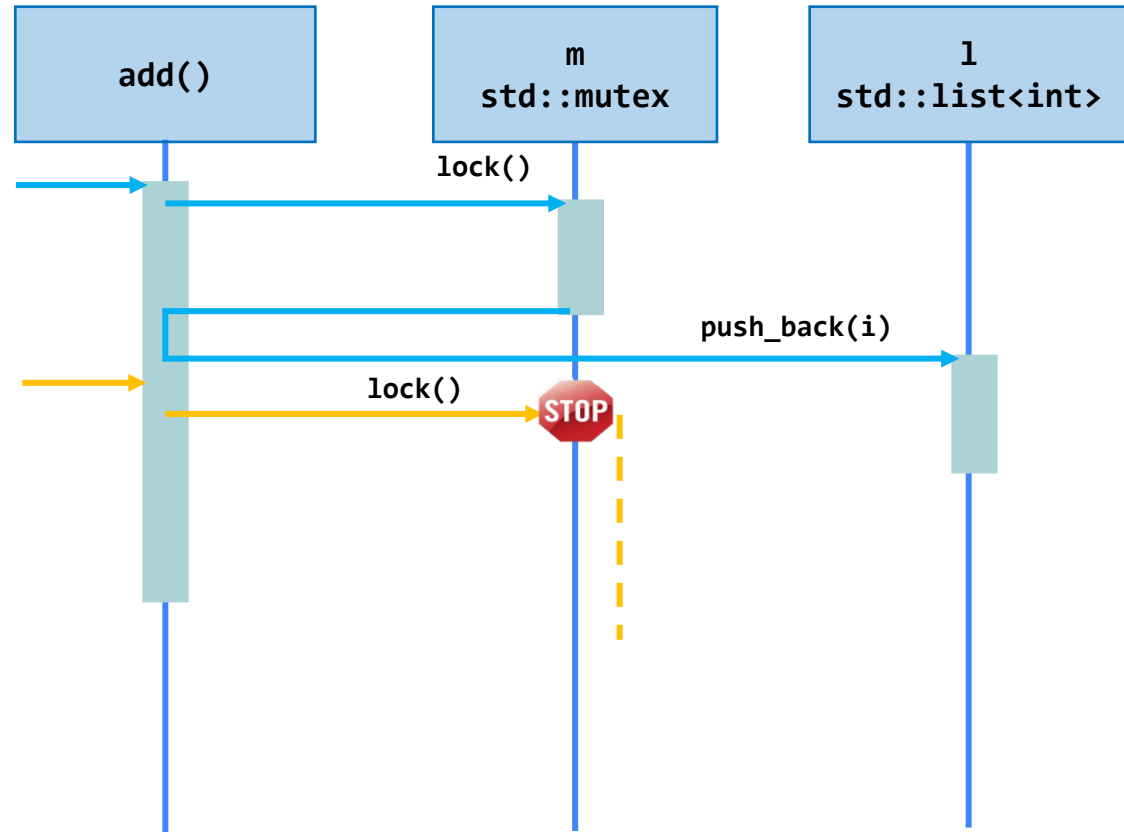


```

std::list<int> l;
std::mutex m;

void add(int i){
    m.lock();
    l.push_back(i);
    m.unlock();
}

```

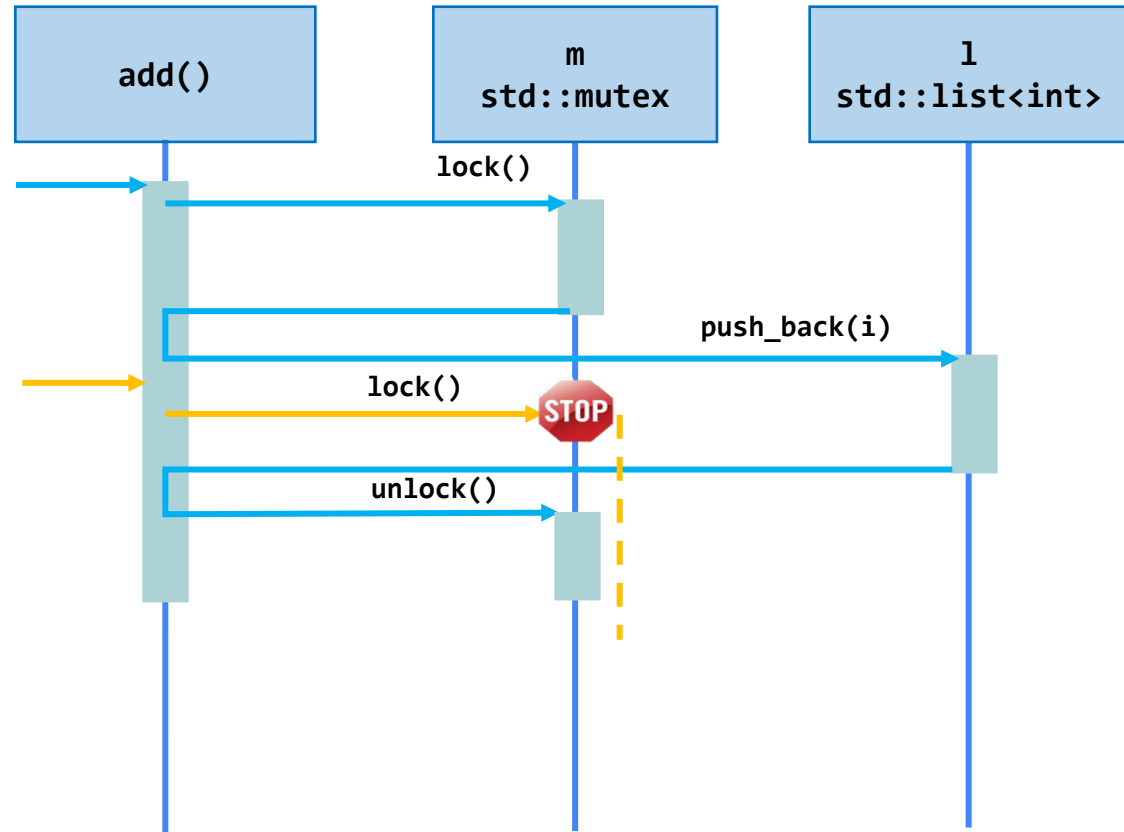



```

std::list<int> l;
std::mutex m;

void add(int i){
    m.lock();
    l.push_back(i);
    m.unlock();
}

```

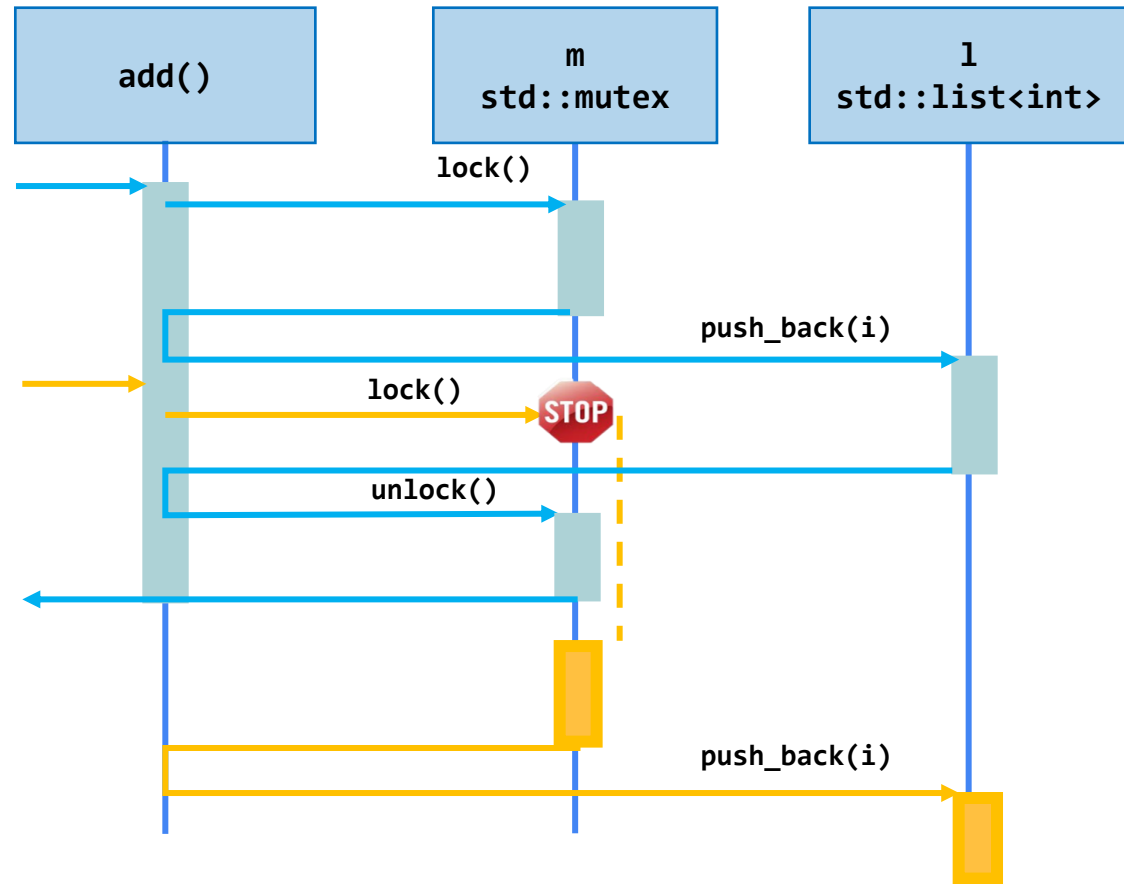


```

std::list<int> l;
std::mutex m;

void add(int i){
    m.lock();
    l.push_back(i);
    m.unlock();
}

```



Rilasciare i mutex

- Se un thread che è in possesso di un mutex termina (anche, ma non solo, per via di un errore) senza rilasciare il mutex, si crea un problema
 - I sistemi operativi tendono liberare il mutex
 - Però, quale stato hanno le risorse che il mutex protegge?
- Per evitare la situazione, in C++ si ricorre al paradigma RAI
 - La classe `std::lock_guard` incapsula un `std::mutex`: il costruttore lo acquisisce, il distruttore lo rilascia; non ci sono altri metodi
 - Questo garantisce che, mentre esiste l'istanza del `lock_guard`, si posseda il `mutex` e se - per qualsiasi motivo - cessa di esistere il `lock_guard`, il `mutex` sia comunque rilasciato

```
namespace std {  
  
    template <class T>  
    class lock_guard {  
  
    private:  
        T& m_lockable;  
  
    public:  
        lock_guard(T& lockable) :  
            m_lockable(lockable) {  
            lockable.lock();  
        }  
  
        ~lock_guard() {  
            m_lockable.unlock();  
        }  
  
    };  
  
}
```

C++

```
template <class T>  
class shared_vector {  
    std::vector<T> v;  
    std::mutex m;  
  
    public:  
        int size() {  
            std::lock_guard<std::mutex> l(m);  
            return v.size();  
        } // il rilascio avviene qui!  
  
        T front() {  
            std::lock_guard<std::mutex> l(m);  
            return v.front();  
        } // il rilascio avviene qui!  
  
        void push_back(T t) {  
            std::lock_guard<std::mutex> l(m);  
            v.push_back(t);  
        } // il rilascio avviene qui!  
  
};
```

C++

Valutazione

- L'uso del pattern RAI garantisce che il lock sia rilasciato automaticamente nel momento in cui l'oggetto **lock_guard** viene distrutto
 - Sia perché l'esecuzione ha raggiunto la normale fine del metodo
 - Sia perché si è verificata un'eccezione e c'è stata una contrazione dello stack
- Tuttavia, dall'uso del pattern **non emerge** quali metodi della classe che contiene il mutex debbano essere **sincronizzati**
 - Né impedisce che venga scritto del codice che fa accesso ai dati condivisi senza possedere il lock
- L'uso corretto della sincronizzazione e le sue evoluzioni nel tempo, legate alla manutenzione della classe, **restano affidate** principalmente **ai commenti** eventualmente presenti nel codice
 - Senza che il compilatore possa fornire un supporto attivo per garantire il rispetto dei vincoli

Algoritmo della Toilette

toilette

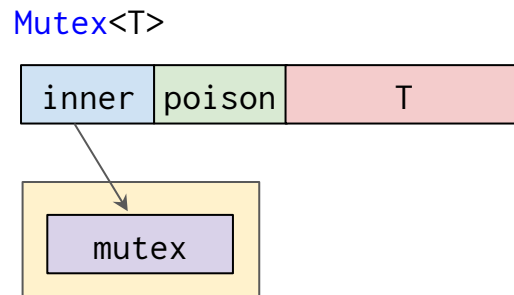


- Chi ha bisogno va alla toilette
- Se è libera entra e si chiude
- Sta nella toilette un tempo ragionevole
- Chi arriva e trova la toilette occupata aspetta in coda
- Quando si finisce di usare la toilette, si pulisce e si esce
- La coda di attesa è all'italiana, uno tra quelli in attesa entra nella toilette.

Mutex in Rust

- L'accesso ad uno stato condiviso in Rust richiede l'utilizzo di **due blocchi** in cascata:
 - Il primo volto a permettere il possesso multiplo di una struttura dati in sola lettura da parte di più thread, realizzato mediante il costrutto `std::sync::Arc<T>`
 - Il secondo che consente l'acquisizione in lettura/scrittura della struttura dati, realizzato alternativamente mediante il costrutto `std::sync::Mutex<T>`, con `std::sync::RwLock<T>` oppure ricorrendo ai **tipi atomici**
- Questa combinazione che prende spunto dal pattern RAI, permette di **rendere esplicito** nella struttura del codice e nei pattern di accesso ai dati **cosa** sia condiviso e **impedisce** di fatto **l'accesso senza** il corretto **possesso** del lock relativo
 - Permettendo al compilatore di bloccare ogni tentativo di accesso non conforme

std::sync::Mutex<T>



- Un oggetto di tipo `Mutex` incapsula un dato di tipo `T` oltre al riferimento ad un mutex nativo del sistema operativo
 - L'unico modo per accedere al dato è invocare il metodo `lock()`
 - Questo metodo restituisce un oggetto di tipo `LockResult<MutexGuard<T>>` e resta bloccato fino a che non è stato possibile acquisire il mutex nativo
 - Se l'ultimo thread che ha acquisito il mutex fosse terminato prima di averlo rilasciato, il mutex si troverebbe nello stato **avvelenato**, e la risposta conterrebbe un errore
- Se il metodo `lock()` ha successo, la risposta contiene un `MutexGuard<T>`
 - Tale oggetto implementa il tratto `Deref<T>` e si comporta come uno smart-pointer
 - Dereferenziandolo, si ottiene un riferimento mutabile al dato `T`
 - Quando il `MutexGuard<T>` esce dallo scope, il mutex nativo viene rilasciato, permettendo ad altri thread di chiederne il possesso
 - A tutti gli effetti `MutexGuard<T>` implementa il pattern RAI, ma - per come viene costruito - è necessariamente disponibile solo se si possiede il mutex

lock()

- Il lock() permette di accedere al dato condiviso e garantisce che la cache venga pulita (invalidata) e dunque si va a leggere il dato direttamente dalla memoria
- Quando si esce dal lock() (con la distruzione del `MutexGuard<T>`) viene effettuata la flush della cache) garantendo che il dato sia trasferito in memoria prima che altri possano leggerlo.

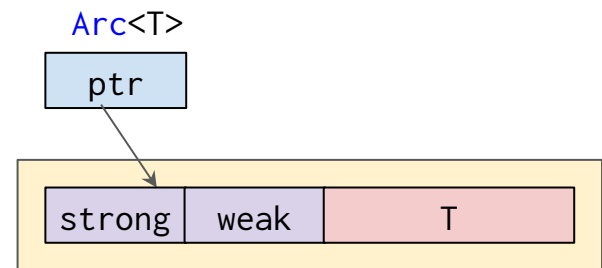
```
use std::sync::Mutex;
use std::thread;
```

```
fn main() {
    let n = Mutex::new(0);
    thread::scope(|s| {
        for _ in 0..10 {
            s.spawn(|| {
                let mut guard = n.lock().unwrap();
                for _ in 0..100 {
                    *guard += 1;
                }

                println!("Alla fine del thread:{:?} n = {:?}",
                        thread::current().id(), guard);
            });
        }
    });
}
```



std::sync::Arc<T>



- Un oggetto di tipo **Mutex** può avere un solo possessore
 - Per superare questo vincolo, lo si incapsula all'interno di un oggetto di tipo **std::sync::Arc<T>**
- **Arc<T>** permette di condividere il possesso di un dato, allocandolo nello heap e mantenendo un conteggio dei riferimenti esistenti di tipo *thread-safe*
 - E' possibile duplicare un oggetto di questo tipo attraverso il metodo **clone()**
 - Tale metodo si limita a duplicare il puntatore al blocco sullo heap, avendo cura di incrementare (in modo atomico) il contatore dei riferimenti associati al dato
 - Il dato clonato viene ceduto ad un thread specificando la parola-chiave `move` di fronte alla funzione lambda che ne descrive la computazione

```
use std::sync::{Arc, Mutex};
use std::thread;
```

```
fn main() {
    let shared_data = Arc::new(Mutex::new(Vec::new()));
    let mut threads = vec![];
    for i in 1..10 {
        let data = shared_data.clone();    //duplicazione del possesso
        threads.push( thread::spawn( move || { //data è ceduto al thread
            let mut v = data.lock().unwrap(); //v è di tipo MutexGuard<T>
            println!("{:?}", i);
            v.push(i);                      //quando v esce dallo scope, il lock
        }));                               //viene rilasciato
    }
    for t in threads { t.join().unwrap(); }

    //v contiene i numeri da 1 a 9
    println!("\nResult: {:?}", *(shared_data.lock().unwrap()));
}
```



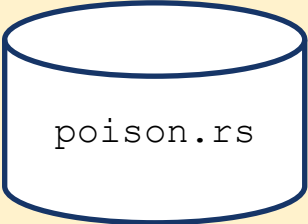
Mutex avvelenato

- Il campo "poison" in un Mutex è un meccanismo per gestire situazioni in cui un thread proprietario di un blocco del Mutex termina in modo anomalo senza rilasciare il blocco. Quando ciò accade, il Mutex entra in uno stato di "veleno" (poisoned), il che significa che il blocco non è stato rilasciato correttamente.
- Questo stato è segnalato al prossimo thread che cerca di ottenere il blocco.
- Quando un MutexGuard tenta di riacquisire il blocco dopo che è stato rilasciato in modo anomalo, il MutexGuard solleva un errore per segnalare la situazione di "poisoning".
- Questo aiuta a identificare e gestire situazioni in cui un thread potrebbe aver lasciato il blocco del Mutex in uno stato inconsistente, aiutando a garantire l'integrità dei dati condivisi.
- E' però possibile accedere al dato protetto dal Mutex (in lettura o scrittura) anche nel caso di Mutex avvelenato attraverso la lettura dell'errore, mediante il metodo `into_error()`.

```

use std::sync::{Arc, Mutex};
use std::thread;
fn main() {
    let data = Arc::new(Mutex::new(0));
    let data_cloned = Arc::clone(&data);
    let _ = thread::spawn(move || {
        let mut num = data_cloned.lock().unwrap();
        *num += 1;
        panic!("Oops! Il thread ha avvelenato il mutex.");
    }).join();
    let result = data.lock(); // Tentiamo di acquisire il mutex nel thread principale
    match result {
        Ok(guard) => { println!("Mutex non avvelenato. Valore: {}", *guard); }
        Err(poisoned) => { // Il mutex è avvelenato. Recuperiamo il valore
            let mut guard = poisoned.into_inner();
            println!("Mutex avvelenato. Valore recuperato: {}", *guard);
            *guard += 1; // Possiamo modificare il dato del mutex
            println!("Stato del mutex resettato. Nuovo valore: {}", *guard);
        }
    }
    let result = data.lock(); // Tentiamo di acquisire nuovamente il mutex nel thread principale
    match result {
        Ok(guard) => { println!("Mutex non avvelenato. Valore: {}", *guard); }
        Err(poisoned) => { // Il mutex rimane avvelenato, ma possiamo ancora recuperare il valore
            let mut guard = poisoned.into_inner();
            println!("Mutex avvelenato. Valore recuperato: {}", *guard);
            *guard += 1; // Possiamo ancora modificare il dato
            println!("Stato del mutex resettato. Nuovo valore: {}", *guard);
        }
    }
}

```

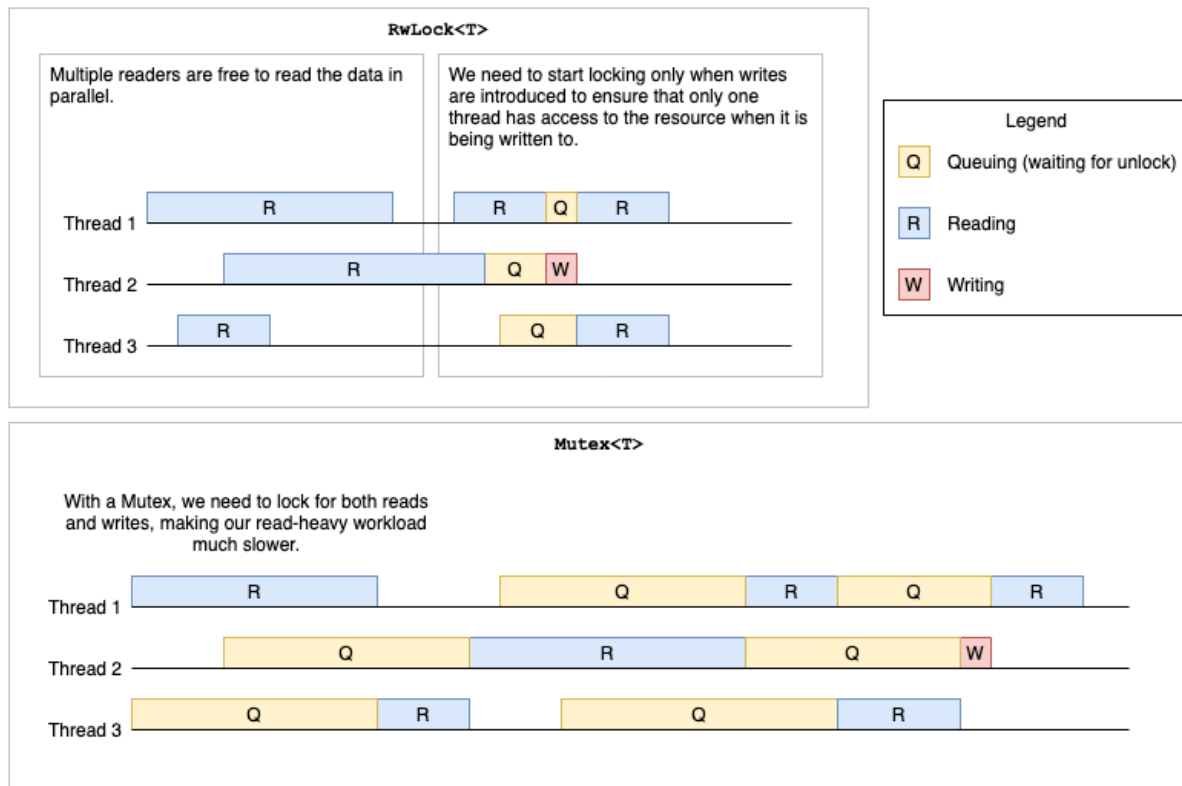


poison.rs



std::sync::RwLock<T>

- Se gli accessi in lettura e in scrittura sono sbilanciati (ossia capita più frequentemente la lettura), può essere conveniente sostituire, alla **struct Mutex<T>**, la **struct RwLock<T>**
 - Questa offre il metodo **read()** per accedere, in modo condiviso, in lettura ed il metodo **write()** per accedere in modo esclusivo in scrittura



std::sync::RwLock<T>

- I metodi `read()` e `write()` restituiscono rispettivamente oggetti di tipo `LockResult<RwLockReadGuard>` e `LockResult<RwLockWriteGuard>`
 - Il risultato contiene un errore se il lock è avvelenato
 - Questo capita se un thread che lo possedeva è terminato senza averlo rilasciato o cercando di acquisirlo ulteriormente
- Entrambi gli oggetti di guardia implementano il pattern RAII
 - Rilasciando il lock nel momento in cui sono distrutti


```
use std::sync::{Arc, RwLock};
use std::thread;

fn main() {
    // Creiamo una struttura dati condivisa protetta da un RwLock
    let data = Arc::new(RwLock::new(vec![1, 2, 3, 4, 5]));

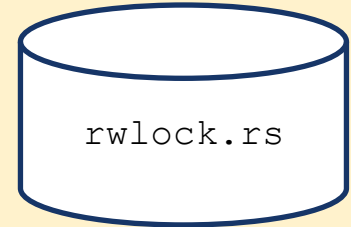
    // Creiamo 10 thread che leggono dalla struttura dati condivisa
    let mut threads = vec![];

    for i in 0..10 {
        // Cloniamo l'arc per ogni thread in modo che ciascuno abbia un riferimento condiviso
        let data_clone = Arc::clone(&data);

        // Creiamo il thread
        let thread = thread::spawn(move || {
            // Otteniamo un blocco di lettura (non esclusivo) sul RwLock
            let guard = data_clone.read().unwrap();
            println!("Thread {}: {:?}", i, *guard);
        });

        threads.push(thread);
    }

    // Attendo che tutti i thread terminino
    for thread in threads {
        thread.join().unwrap();
    }
}
```



```
use std::sync::{Arc, RwLock};  
use std::thread;
```

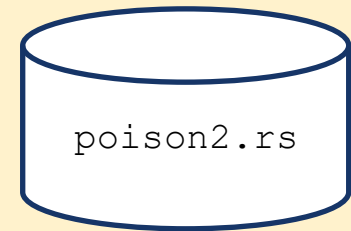
```
fn main() {  
    // Creiamo una struttura dati condivisa protetta da un RwLock  
    let data = Arc::new(RwLock::new(vec![1, 2, 3]));  
  
    // Cloniamo l'arc per ogni thread in modo che ciascuno abbia un riferimento condiviso  
    let data_clone1 = Arc::clone(&data);  
    let data_clone2 = Arc::clone(&data);  
  
    // Thread che legge dalla struttura dati condivisa  
    let reader = thread::spawn(move || {  
        // Otteniamo un blocco di lettura (non esclusivo) sul RwLock  
        let guard = data_clone1.read().unwrap();  
        println!("Thread lettore: {:?}", guard);  
    });  
  
    // Thread che scrive sulla struttura dati condivisa  
    let writer = thread::spawn(move || {  
        // Otteniamo un blocco di scrittura (esclusivo) sul RwLock  
        let mut guard = data_clone2.write().unwrap();  
        guard.push(4);  
        println!("Thread scrittore: {:?}", guard);  
    });  
  
    // Attendo che entrambi i thread terminino  
    reader.join().unwrap();  
    writer.join().unwrap();  
}
```



```
use std::time::Duration;
use std::sync::{Arc, RwLock};
use std::thread;

fn main() {
    let data = Arc::new(RwLock::new(vec![1, 2, 3]));
    let data_clone1 = Arc::clone(&data);
    let data_clone2 = Arc::clone(&data);
    let data_clone3 = Arc::clone(&data);
    let reader1 = thread::spawn(move || { // Thread in lettura 1
        let guard = data_clone1.read().unwrap();
        println!("Thread lettura 1: {:?}", *guard);
    });
    let reader2 = thread::spawn(move || { // Thread in lettura 2
        thread::sleep(Duration::from_secs(1));
        let guard = data_clone2.read().unwrap();
        // la lettura di un RwLock avvelenato causa un panic
        println!("Thread lettura 2: {:?}", *guard);

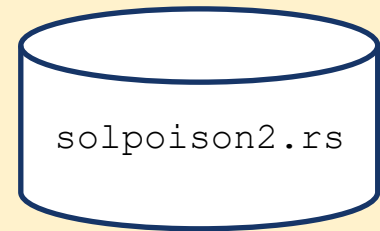
    });
    let writer = thread::spawn(move || { // Thread in scrittura (che provoca uno stato "poisoned")
        let mut guard = data_clone3.write().unwrap();
        guard.push(4); // Prova a inserire un elemento nella struttura dati
        panic!("Oops, ho fatto un errore!"); // Simula un panic
    });
    reader1.join().unwrap();
    reader2.join().unwrap_err();
    writer.join().unwrap_err();
    println!("Fine");
}
```



```

use std::time::Duration;
use std::sync::{Arc, RwLock};
use std::thread;
fn main() {
    let data = Arc::new(RwLock::new(vec![1, 2, 3]));
    let data_clone1 = Arc::clone(&data);
    let data_clone2 = Arc::clone(&data);
    let data_clone3 = Arc::clone(&data);
    let reader1 = thread::spawn(move || { // Thread in lettura 1
        let guard = data_clone1.read().unwrap();
        println!("Thread lettura 1: {:?}", *guard);
    });
    let reader2 = thread::spawn(move || { // Thread in lettura 2
        thread::sleep(Duration::from_secs(1));
        let guard = data_clone2.read();
        match guard {
            Ok(guard) => { println!("Valore letto con successo: {:?}", guard); }
            Err(poison_error) => {
                println!("Reader 2: RwLock è stato avvelenato. Contiene {:?}", poison_error.into_inner());
            }
        }
    });
    let writer = thread::spawn(move || { // Thread in scrittura (che provoca uno stato "poisoned")
        let mut guard = data_clone3.write().unwrap();
        guard.push(4); // Prova a inserire un elemento nella struttura dati
        panic!("Oops, ho fatto un errore!"); // Simula un errore (ad esempio, un panic)
    });
    reader1.join().unwrap();
    reader2.join().unwrap();
    writer.join().unwrap_err();
}

```



Tipi atomici

- Il modulo `std::sync::atomic` mette a disposizione alcune strutture dati che costituiscono primitive di comunicazione tra thread basate sul principio della memoria condivisa
 - Esso offre versioni atomiche di valori booleani, numeri interi con e senza segno e puntatori nativi (`AtomicBool`, `AtomicIsize`, `AtomicUsize`, `AtomicI8`, `AtomicU8`, `AtomicI16`, `AtomicU16`, `AtomicI32`, `AtomicU32`, `AtomicI64`, `AtomicU64`)
 - Ciascun tipo è associato ad operazioni che, se usate correttamente, permettono di sincronizzare gli aggiornamenti di tali valori (*read and modify*) tra thread differenti
 - L'operazione di aggiornamento diventa indivisibile, evitando *undefined behavior*

Tipi atomici: load e store

- Permettono operazioni di lettura (**load**) e scrittura (**store**) con associata barriera di memoria

```
impl AtomicI32 {  
    pub fn load(&self, ordering: Ordering) -> i32;  
    pub fn store(&self, value: i32, ordering: Ordering);  
}
```

Ordering

- Ogni operazione atomica definisce un argomento di tipo `std::sync::atomic::Ordering` che specifica come le operazioni atomiche sincronizzano la memoria tra i thread. Le possibili varianti sono:
 - Relaxed: Questa è la variante più debole. Non ha vincoli di ordinamento
 - Release: applicabile solo per operazioni che eseguono una **scrittura** e determina che tutte le operazioni non atomiche eseguite prima dell'operazione atomica **non** vengano riordinate **dopo** di essa
 - Acquire: applicabile solo per operazioni che eseguono una **lettura** e determina che tutte le operazioni non atomiche eseguite dopo l'operazione atomica **non** vengano riordinate **prima** di essa
 - AcqRel: applicabile solo per operazioni che combinano sia **letture** che **scritture** e garantisce sia l'acquire che il release (nessuna operazione non atomica che precede l'istruzione atomica è riordinata dopo di essa e nessuna operazione non atomica che segue l'istruzione è riordinata prima di essa)
 - SeqCst: impone che tutti i thread vedono tutte le operazioni sequenzialmente consistenti nello stesso ordine.

```

fn main() {
    let buffer = Arc::new(Mutex::new(Vec::with_capacity(10)));
    let buffer_clone1 = Arc::clone(&buffer);
    let buffer_clone2 = Arc::clone(&buffer);
    let producer_finished = Arc::new(AtomicBool::new(false));
    let producer_finished_clone = Arc::clone(&producer_finished);
    let producer_handle = thread::spawn(move || {
        for i in 0..10 {
            // thread::sleep(std::time::Duration::from_nanos(10));
            let mut buffer = buffer_clone1.lock().unwrap();
            buffer.push(i); // Aggiungi un valore al buffer
            println!("Produttore: prodotto {}", i);
        }
        producer_finished_clone.store(true, Ordering::Release);
    });
    let consumer_handle = thread::spawn(move || {
        loop {
            let mut buffer = buffer_clone2.lock().unwrap();
            let len = buffer.len();
            if len > 0 {
                let value = buffer.remove(0); // Se ci sono elementi nel buffer, consumane uno
                println!("Consumatore: consumato {}", value);
            } else if producer_finished.load(Ordering::Acquire) {
                break; // Se il produttore ha finito, esce dal ciclo
            } // rilascia il lock e attendi finché ci sono elementi nel buffer o il produttore ha finito
        }
        println!("Tutti gli elementi sono stati consumati");
    });
    producer_handle.join().unwrap();
    consumer_handle.join().unwrap();
}

```



Come fa a sapere il consumatore
che il produttore ha finito di
produrre?

Tipi atomici: `fetch_modify`

- Funzionalità di tipo Read-Modify-Write: modificano la variabile atomica, ma anche leggono (*fetch*) il valore originale in una singola istruzione atomica

```
impl AtomicI32 {  
    pub fn fetch_add(&self, v: i32, ordering: Ordering) -> i32;  
    pub fn fetch_sub(&self, v: i32, ordering: Ordering) -> i32;  
    pub fn fetch_or(&self, v: i32, ordering: Ordering) -> i32;  
    pub fn fetch_and(&self, v: i32, ordering: Ordering) -> i32;  
    pub fn fetch_nand(&self, v: i32, ordering: Ordering) -> i32;  
    pub fn fetch_xor(&self, v: i32, ordering: Ordering) -> i32;  
    pub fn fetch_max(&self, v: i32, ordering: Ordering) -> i32;  
    pub fn fetch_min(&self, v: i32, ordering: Ordering) -> i32;  
    pub fn swap(&self, v: i32, ordering: Ordering) -> i32; // "fetch_store"  
}
```

```
use std::sync::atomic::Ordering::Relaxed;
```

```
fn main() {
```

```
    use std::sync::atomic::AtomicI32;
```

```
    let a = AtomicI32::new(100);
```

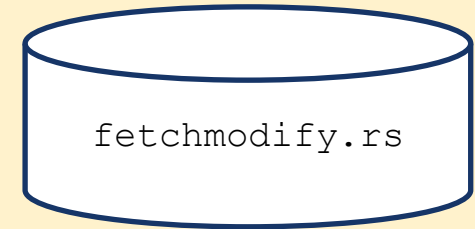
```
    let b = a.fetch_add(23, Relaxed);
```

```
    let c = a.load(Relaxed);
```

```
    assert_eq!(b, 100);
```

```
    assert_eq!(c, 123);
```

```
}
```



Tipi atomici: compare_exchange

- Funzionalità di tipo Read-Modify-Write: Questa operazione controlla, atomicamente, se il valore atomico è uguale a un determinato valore e solo in tal caso lo sostituisce con un nuovo valore,
- Restituirà il valore precedente e ci dirà se lo ha sostituito o meno.

```
impl AtomicI32 {  
    pub fn compare_exchange(&self, expected: i32, new: i32)  
        -> Result<i32, i32> {  
        let v = self.load();  
        if v == expected {  
            // Value is as expected.  
            // Replace it and report success.  
            self.store(new);  
            Ok(v)  
        } else { // The value was not as expected.  
            // Leave it untouched and report failure.  
            Err(v)  
        }  
    }  
}
```

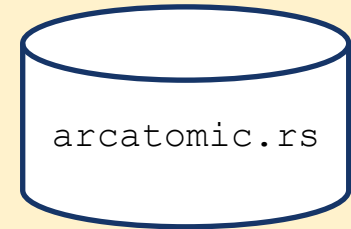
Tipi atomici

- Sebbene siano tutti *thread-safe* (implementano il tratto **Sync**), non offrono meccanismi di condivisione esplicita
 - Come tutti i valori in Rust, sono soggetti alla regola del possessore unico
 - Per permettere a più thread di accedere al loro valore, è comune
 - incapsularli all'interno di un elemento di tipo **Arc<T>** oppure
 - dichiararli come **variabili globali**, attraverso la parola chiave **static**
- implementano il meccanismo di mutabilità interna (in modo analogo a **Cell<T>**)
 - Poiché le operazioni di modifica sono garantite essere *thread-safe*, i metodi che ne modificano il contenuto richiedono solo un accesso condiviso (**&self**) e non un accesso esclusivo (**&mut self**)

```
use std::sync::Arc;
use std::sync::atomic::{AtomicUsize, Ordering};
use std::thread;

fn main() {
    const NUM_THREADS: usize = 4;
    const NUM_INCREMENTS: usize = 100_000;

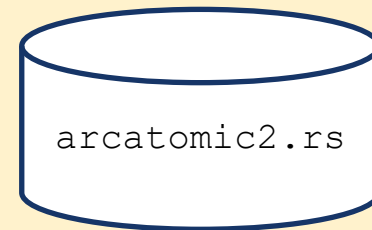
    let counter = Arc::new(AtomicUsize::new(0));
    let mut handles = vec![];
    for _ in 0..NUM_THREADS {
        let counter = Arc::clone(&counter);
        let handle = thread::spawn(move || {
            for _ in 0..NUM_INCREMENTS {
                counter.fetch_add(1, Ordering::Relaxed);
            }
        });
        handles.push(handle);
    }
    for handle in handles {
        handle.join().unwrap();
    }
    println!("Final counter value: {}", counter.load(Ordering::Relaxed) );
}
```



```
use std::sync::Arc;
use std::thread;
use std::sync::atomic::{AtomicBool, Ordering};
fn main() {
    let running = Arc::new(AtomicBool::new(true));
    let running_clone = Arc::clone(&running);
    let handle = thread::spawn(move || {
        while running_clone.load(Ordering::Relaxed) {
            println!("Working...");
            thread::sleep(std::time::Duration::from_secs(1));
        }
        println!("Thread exiting...");
    });
    // Simulate main thread doing some work
    thread::sleep(std::time::Duration::from_secs(5));

    // Set the running flag to false to signal the thread to exit
    running.store(false, Ordering::Relaxed);

    // Wait for the spawned thread to finish
    handle.join().unwrap();
}
```



Variabili Globali

- **Le variabili globali possono essere solo immutabili**
- Variabili globali mutabili possono essere solo inserite in blocchi unsafe.

```
static mut COUNTER:i32 = 0;

fn increment_counter() {
    COUNTER += 1;
    println!("New value of COUNTER: {}", COUNTER);
}

fn main() {
    for _ in 1..10 {
        increment_counter();
    }
    println!("Final value of COUNTER: {}", COUNTER);
}
```



```
error[E0133]: use of mutable static is unsafe and requires unsafe function or block
--> src/main.rs:5:5
```

```
5 | | COUNTER += 1;
  | | ^^^^^^^ use of mutable static
```

```
use std::sync::atomic::{AtomicI32, Ordering};

static COUNTER: AtomicI32 = AtomicI32::new(0);

fn increment_counter() {
    COUNTER.fetch_add(1, Ordering::Relaxed);
    // Incrementa il contatore in modo sicuro
    println!("Valore aggiornato di COUNTER: {}", COUNTER.load(Ordering::Relaxed));
}

fn main() {
    for _ in 1..10 {
        increment_counter();
    }
    println!("Valore finale di COUNTER: {}", COUNTER.load(Ordering::Relaxed));
}
```




```
use std::sync::atomic::{AtomicUsize, Ordering};
use std::thread;

static COUNTER: AtomicUsize = AtomicUsize::new(0);

fn update_counter(new_value: usize) {
    let expected_value = 0;
    // Provare a impostare il valore di counter sul nuovo valore
    if let Err(actual_value) = COUNTER.compare_exchange(expected_value, new_value,
Ordering::Relaxed,
    Ordering::Relaxed) {
        panic!("Fallimento in compare_exchange(): contiene {} anziché {}", actual_value,
expected_value);
    }
}

fn main() {
    let thread = thread::spawn(move || {
        update_counter(1);
    });
    thread.join().unwrap();

    println!("Final value of COUNTER: {}", COUNTER.load(Ordering::Relaxed) );
}
```



```
const SOGLIA: usize = 100;
fn main() {
    let num_done = AtomicUsize::new(0);
    thread::scope(|s| {
        // A background thread to process all 100 items
        s.spawn(|| {
            for i in 0..SOGLIA {
                //println!("{}", i);
                thread::sleep(Duration::from_nanos(1));
                num_done.store(i + 1, Relaxed);
            }
        });
        // The main thread shows status updates, every 10 nanoseconds.
        loop {
            let n = num_done.load(Relaxed);
            if n == SOGLIA {break;}
            println!("Working.. {n}/100 done");
            thread::sleep(Duration::from_nanos(10));
        }
    });
    println!("Done!");
}
```

Perché non devo
incapsularlo in un Arc
e non l'ho definito
static, ma funziona?

CHALLENGE

atomic.rs

std::sync::Weak<T>

- Analogamente a quanto succede con gli smart pointer di tipo **Rc<T>**, anche nel caso di **Arc<T>** la creazione di catene circolari impedisce il rilascio delle strutture
 - Per questo motivo è disponibile la **struct std::sync::Weak<T>** che permette - sulla falsariga di quanto avviene con **std::rc::Weak<T>** - di realizzare dipendenze circolari con riferimenti che non partecipano al conteggio, garantendo così la possibilità di rilascio
 - Per fare accesso al dato puntato, occorre invocare il metodo **upgrade()**, che restituisce un valore di tipo **Option<Arc<T>>**
- Si crea un oggetto di tipo **Weak<T>** a partire da un riferimento di tipo **Arc<T>** invocando su quest'ultimo il metodo **downgrade()**

```
fn main() {  
    let data = Arc::new("hello".to_string());  
    let weak_ref = Arc::downgrade(&data); // Create a weak reference  
  
    // Try to upgrade the weak reference  
    if let Some(upgraded) = weak_ref.upgrade() {  
        println!("Weak reference still alive: {}", upgraded.to_uppercase());  
    } else {  
        println!("Weak reference is no longer valid.");  
    }  
  
    // Dropping the strong reference  
    drop(data);  
  
    // Attempt to upgrade again (should return None)  
    if let Some(upgraded) = weak_ref.upgrade() {  
        println!("Weak reference still alive: {}", upgraded);  
    } else {  
        println!("Weak reference is no longer valid.");  
    }  
}
```



Attese condizionate

- Spesso un thread deve aspettare un risultato intermedio prodotto da altri thread e, per motivi di efficienza, l'attesa non deve consumare risorse e deve terminare non appena un dato è disponibile
- Una soluzione potrebbe basarsi sul polling del valore del risultato
- La presenza di dati condivisi richiede come minimo l'utilizzo di un mutex per garantire l'assenza di interferenze tra i due thread che devono fare accesso ai dati
- Il polling ha due limiti
 - Consuma capacità di calcolo e batteria in cicli inutili
 - Introduce una latenza tra il momento in cui il dato è disponibile e il momento in cui il secondo thread si sblocca
- Per gestire queste situazioni, i sistemi operativi offrono il concetto di **condition variable**
 - Strutture dati di sincronizzazione che permettono di bloccare l'esecuzione di un thread, così da evitare il consumo di CPU, nell'attesa che qualcosa succeda

Attese condizionate

- L'uso di una **condition variable** è basata sulla cooperazione all'interno del sistema:
 - se un thread si sospende in attesa di una condizione, è necessario che tutti i thread che eseguono azioni che potrebbero provocare il verificarsi della condizione si facciano carico di inviare una notifica alla condition variable
- Il pattern di utilizzo prevede che esista una espressione booleana il cui valore possa essere usato per determinare se occorre attendere o meno
 - La valutazione di tale espressione deve avvenire mentre si possiede un mutex, per garantire l'assenza di corse critiche
 - In Rust, questo vuol dire che le variabili che consentono la valutazione della condizione sono incapsulate nel mutex
- Ogni **condition variable** deve essere usata in coppia con un singolo mutex
 - Eventuali tentativi di usare mutex diversi per una stessa *condition variable* può determinare un fallimento in fase di esecuzione

Attese condizionate

- Il linguaggio C++ offre la classe **`std::condition_variable`**
 - Essa viene usata in coppia con un oggetto di tipo **`std::mutex`** racchiuso all'interno di un oggetto di tipo **`std::unique_lock<std::mutex>`**
- Rust offre la struct **`std::sync::Condvar`**
 - La sua semantica è totalmente allineata con la corrispondente classe C++
 - La struttura dei suoi metodi facilita il collegamento con il dato protetto dal mutex, rendendo più naturale il suo utilizzo

Condvar - metodi principali

- **pub fn new() -> Condvar**
 - Crea una nuova istanza
- **pub fn wait<'a, T>(&self, guard: MutexGuard<'a, T>) -> LockResult<MutexGuard<'a, T>>**
 - Sospende il thread corrente fino alla ricezione di una notifica: durante la sospensione, rilascia il lock; al ricevere della notifica, riacquisisce il lock e restituisce una nuova guardia
- **pub fn notify_one(&self)**
 - Sveglia un thread a caso tra quelli in attesa sulla condition variable
- **pub fn notify_all(&self)**
 - Sveglia tutti i thread in attesa sulla condition variable, che usciranno, uno alla volta, dal metodo wait possedendo il lock

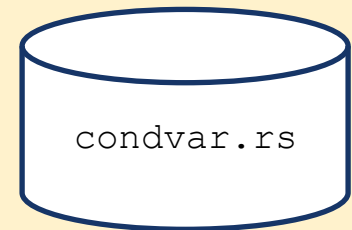

```
use std::sync::{Arc, Mutex, Condvar};
use std::thread;
use std::time::Duration;

fn main() {
    // Create an Arc pair containing a Mutex and a Condvar
    let pair = Arc::new( (Mutex::new(false), Condvar::new()) );
    let pair2 = Arc::clone(&pair);

    // Inside our lock, spawn a new thread and wait for it to start
    thread::spawn(move || {
        let (mutex, cvar) = &*pair2;
        let mut started = mutex.lock().unwrap();
        thread::sleep(Duration::from_secs(5));
        *started = true; // We notify the Condvar that the value has changed
        cvar.notify_one();
    });

    // Wait for the thread to start up
    let (mutex, cvar) = &*pair;
    let mut started = mutex.lock().unwrap();

    println!("Waiting ...");
    started = cvar.wait(started).unwrap();
    println!("Thread started!");
}
```



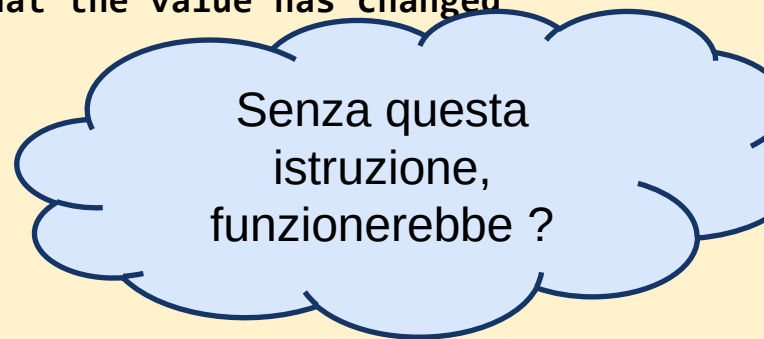
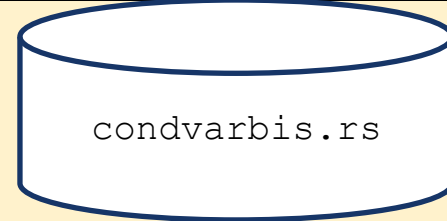
```
use std::sync::{Arc, Mutex, Condvar};
use std::thread;
use std::time::Duration;
```

```
fn main() {
    // Create an Arc pair containing a Mutex and a Condvar
    let pair = Arc::new( (Mutex::new(false), Condvar::new()) );
    let (mutex, cvar) = &*pair;
    let pair2 = Arc::clone(&pair);

    // Inside our lock, spawn a new thread and wait for it to start
    thread::spawn(move || {
        let (mutex, cvar) = &*pair2;
        let mut started = mutex.lock().unwrap();
        thread::sleep(Duration::from_secs(5));
        //*started = true; // We notify the Condvar that the value has changed
        cvar.notify_one();
    });

    // Wait for the thread to start up

    let mut started = mutex.lock().unwrap();
    println!("Waiting ...");
    started = cvar.wait(started).unwrap();
    println!("Thread started!");
}
```



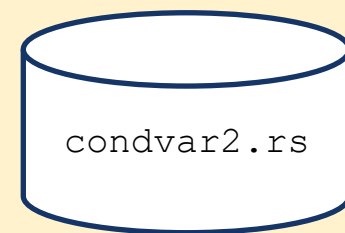
Meccanismo di funzionamento

- Concettualmente, una condition variable mantiene una collezione di thread in attesa che si verifichi la condizione attesa
 - Inizialmente la lista è vuota
 - Quando un thread esegue il metodo `wait(...)`, viene sospeso e aggiunto alla lista
- Quando sono eseguiti i metodi `notify_one()` o `notify_all()`, uno o tutti i thread presenti nella collezione sono risvegliati
 - Si basa sul S.O. per sospendere/risvegliare i thread
- La presenza di un unico lock fa sì che, se più thread ricevono la notifica, il risveglio sia progressivo
 - Non appena un thread rilascia il lock, un altro può acquisirlo e proseguire
- La relazione tra l'evento e la notifica è solo nella testa del programmatore
 - Per questo, si rende esplicito l'evento che si è verificato appoggiandosi ad una o più variabili condivise (sotto il controllo del mutex)

```

use std::sync::{Arc, Mutex, Condvar};
use std::thread;
fn main() {
    const NUM_THREADS: usize = 5;
    let data = Arc::new((Mutex::new(0), Condvar::new()));
    let mut handles = vec![];
    for i in 0..NUM_THREADS {
        let data_clone = Arc::clone(&data);
        let handle = thread::spawn(move || {
            let (mutex, cvar) = &*data_clone;
            let mut data_guard = mutex.lock().unwrap();
            println!("Thread {} in attesa...", i);
            // Attendi fino a quando il dato non è diverso da 0
            if *data_guard == 0 { // dopo capiremo che e' meglio avere while come e' nella versione che trovate in rete
                data_guard = cvar.wait(data_guard).unwrap();
            }
            println!("Thread {} svegliato! Il dato è: {}", i, *data_guard);
        });
        handles.push(handle);
    }
    // Facciamo avanzare i thread dopo 2 secondi
    thread::sleep(std::time::Duration::from_secs(2));
    // Cambiamo il dato condiviso e svegliamo tutti i thread
    {
        let (mutex, cvar) = &*data;
        let mut data_guard = mutex.lock().unwrap();
        *data_guard = 42;
        cvar.notify_all();
    }
    for handle in handles {
        handle.join().unwrap();
    }
}

```



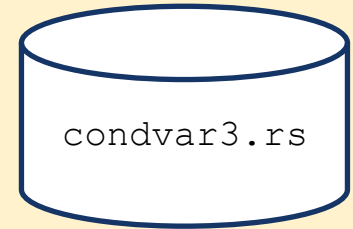
Notifiche spurie

- È possibile che un thread in attesa su una condition variable sia risvegliato in assenza di un'esplicita notifica
 - Problema delle cosiddette **notifiche spurie**
- Occorre, al ritorno dal metodo **wait()**, controllare se la condizione attesa è verificata
 - Per semplificare tale verifica, esiste una versione del metodo di attesa che riceve come argomento una funzione volta a valutare il predicato richiesto
- **pub fn wait_while<'a, T, F>(
 &self,
 guard: MutexGuard<'a, T>,
 condition: F
)
-> LockResult<MutexGuard<'a, T>>
where F: FnMut(&mut T) -> bool**
 - Al risveglio, ri-acquisisce il lock e valuta la funzione **condition**: se questa restituisce **true**, si riaddormenta, altrimenti esce dall'attesa

```

use std::{
    sync::{Arc, Condvar, Mutex},
    thread::{sleep},
    time::Duration,
};
struct Counter {
    value: Mutex<u32>,
    condvar: Condvar,
}
fn main() {
    let counter = Arc::new(Counter {
        value: Mutex::new(0),
        condvar: Condvar::new(),
    });
    let counter_clone = counter.clone();
    let counting_thread = std::thread::spawn(move || loop {
        sleep(Duration::from_millis(100));
        let mut value = counter_clone.value.lock().unwrap();
        *value += 1;
        if *value >= 15 {
            counter_clone.condvar.notify_all();
            break;
        }
    });
    // Wait until the value more or equal to 15
    let mut value = counter.value.lock().unwrap();
    value = counter.condvar.wait_while(value, |val| *val < 15).unwrap();
    println!("Condition met. Value is now {}", *value);
    // Wait for counting thread to finish
    counting_thread.join().unwrap();
}

```



Notifiche perse

- Analogamente, se un thread ha eseguito una qualche azione che può abilitare la prosecuzione di un altro thread, ed invoca il metodo `notify_one()` / `notify_all()` per segnalare tale fatto, è possibile che la notifica vada persa
 - Succede se l'altro thread **non ha ancora eseguito** la corrispondente istruzione di attesa
- Per questo motivo, occorre sempre racchiudere l'istruzione di attesa in un ciclo che verifica se occorra o meno addormentarsi e, al risveglio, se ci siano le condizioni o meno per continuare a dormire
 - In entrambi i casi, il metodo `wait_while(...)` protegge
 - Esso infatti è equivalente al seguente blocco di codice:

```
while condition(&mut *guard) {  
    guard = self.wait(guard)?;  
}  
Ok(guard)
```

```

use std::sync::{Arc, Mutex, Condvar};
use std::thread;
use std::time::Duration;

fn main() {
    // Create an Arc pair containing a Mutex and a Condvar
    let pair = Arc::new( (Mutex::new(false), Condvar::new()) );
    let pair2 = Arc::clone(&pair);

    // Inside our lock, spawn a new thread and wait for it to start
    thread::spawn(move || {
        let (mutex, cvar) = &*pair2;
        let mut started = mutex.lock().unwrap();
        //thread::sleep(Duration::from_secs(1));
        *started = true; // We notify the Condvar that the value has changed
        cvar.notify_one();
    });

    // Wait for the thread to start up
    let (mutex, cvar) = &*pair;
    println!("Waiting ...");
    thread::sleep(Duration::from_secs(1));
    let mut started = mutex.lock().unwrap();
    started = cvar.wait(started).unwrap();
    println!("Thread started! {:?}" , *started);
}

```




```

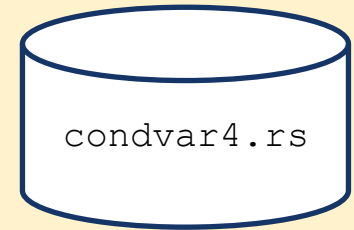
use std::sync::{Arc, Mutex, Condvar};
use std::thread;
use std::time::Duration;

fn main() {
    // Create an Arc pair containing a Mutex and a Condvar
    let pair = Arc::new( (Mutex::new(false), Condvar::new()) );
    let pair2 = Arc::clone(&pair);

    // Inside our lock, spawn a new thread and wait for it to start
    thread::spawn(move || {
        let (mutex, cvar) = &*pair2;
        let mut started = mutex.lock().unwrap();
        //thread::sleep(Duration::from_secs(1));
        *started = true; // We notify the Condvar that the value has changed
        cvar.notify_one();
    });

    // Wait for the thread to start up
    let (mutex, cvar) = &*pair;
    println!("Waiting ...");
    thread::sleep(Duration::from_secs(1));
    let mut started = mutex.lock().unwrap();
    while !*started {
        started = cvar.wait(started).unwrap();
    }
    println!("Thread started! {:?}, *started);
}

```



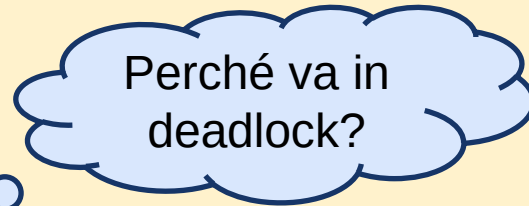
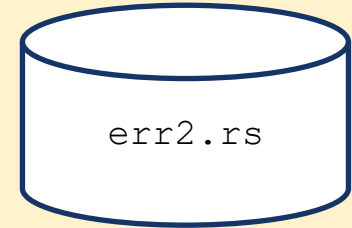
```

use std::{
    sync::{Arc, Condvar, Mutex},
    thread::{sleep},
    time::Duration,
};

struct Counter {
    value: Mutex<u32>,
    condvar: Condvar,
}

fn main() {
    let counter = Arc::new(Counter {
        value: Mutex::new(0),
        condvar: Condvar::new(),
    });
    let counter_clone = counter.clone();
    let counting_thread = std::thread::spawn(move || loop {
        sleep(Duration::from_millis(100));
        let mut value = counter_clone.value.lock().unwrap();
        *value += 1;
        counter_clone.condvar.notify_all();
        if *value > 15 {
            break;
        }
    });
    // Wait until the value more or equal to 15
    let mut value = counter.value.lock().unwrap();
    value = counter.condvar.wait_while(value, |val| *val < 15).unwrap();
    println!("Condition met. Value is now {}. ", *value);
    // Wait for counting thread to finish
    counting_thread.join().unwrap();
}

```



Attesa temporizzata

- Altri metodi permettono di limitare il tempo massimo di attesa, permettendo al thread di risvegliarsi anche in assenza del verificarsi della condizione, ma a seguito dello scadere di un time-out
- Come `wait`, il lock specificato verrà riacquisito all'uscita della funzione `wait_timeout*`
- Restituisce una tupla, in cui il primo campo è un `MutexGuard` e permette di verificare se la condizione si è verificata e il secondo campo fornisce un'informazione sulla condizione di time-out.

- ```
pub fn wait_timeout<'a, T>(
 &self,
 guard: MutexGuard<'a, T>,
 dur: Duration
) -> LockResult< (MutexGuard<'a, T>, WaitTimeoutResult) >
```

- Attende per un tempo massimo pari a `Duration`

- ```
pub fn wait_timeout_while<'a, T, F>(
    &self,
    guard: MutexGuard<'a, T>,
    dur: Duration,
    condition: F
) -> LockResult< (MutexGuard<'a, T>, WaitTimeoutResult) >
where F: FnMut(&mut T) -> bool
```

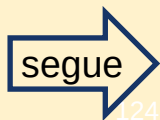
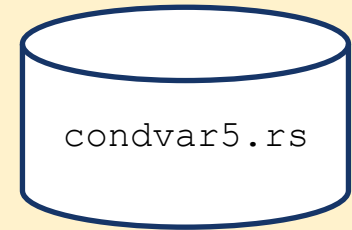
- Attende per un tempo massimo pari a `Duration`; eventuali notifiche ricevute portano a ri-addormentarsi se la funzione `condition` restituisce false

```

use std::sync::{Arc, Mutex, Condvar};
use std::thread;
use std::time::Duration;
struct SharedData{
    mutex:Mutex<bool>,
    cv:Condvar
}
impl SharedData{
    //Metodo costruttore
    pub fn new(condition:bool)->Self{
        SharedData { mutex:Mutex::new(condition), cv:Condvar::new() }
    }
    pub fn change_and_notify(&self){
        let mut data=self.mutex.lock().unwrap();
        *data = true;
        //Mandiamo la notifica alla condvar
        self.cv.notify_one();
    }
    pub fn looper(&self){
        loop {
            //Il thread aspetta per una notifica. Nel caso siano passati 100 misecondi smette di aspettare
            let lock: (MutexGuard<bool>, WaitTimeoutResult) = self.cv.wait_timeout(
                self.mutex.lock().unwrap(), Duration::from_millis(100)).unwrap();

            if *lock.0 == true {
                //Il thread ha ricevuto una notifica dato che il valore è stato cambiato, quindi esco
                println!("Il valore è cambiato quindi posso uscire dal ciclo correttamente ");
                if !lock.1.timed_out() {
                    println!("e non c'è stato time-out");
                }
                break
            }
            if lock.1.timed_out() {
                println!("E' scaduto il timer ma il valore non è cambiato. Ricomincio il ciclo");
            }
        }
    }
}

```



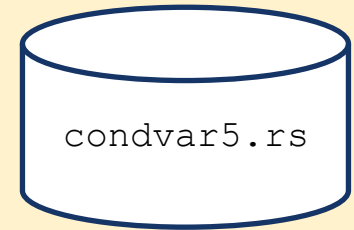
```
fn main(){
    let shared = Arc::new(SharedData::new(false));
    let shared2 = Arc::clone(&shared);

    let mut handles = vec![]; //vettore dei threads creati per poi fare le dovute join

    handles.push(thread::spawn(move|| {
        shared2.looper();
    }));

    handles.push(thread::spawn(move|| {
        //Aspetto prima di mandare la notifica
        thread::sleep(Duration::from_secs(1));
        shared.change_and_notify();
    }));

    //Join finali
    for handle in handles {
        handle.join().expect("Error");
    }
}
```



```

fn main() {
    let pair = Arc::new((Mutex::new(false), Condvar::new()));
    let pair2 = Arc::clone(&pair);

    thread::spawn(move || {
        let (lock, cvar) = &*pair2;
        thread::sleep(Duration::from_secs(2));
        let mut pending = lock.lock().unwrap();
        *pending = true;
        // We notify the condvar that the value has changed.
        cvar.notify_one();
    });

    let (lock, cvar) = &*pair;
    loop {
        let result = cvar.wait_timeout_while(
            lock.lock().unwrap(),
            Duration::from_millis(500),
            |&mut pending| !pending,
        ).unwrap();
        if *result.0 {
            println!("OK");
            break;
        }
        if result.1.timed_out() {
            // timed-out without the condition ever evaluating to false.
            println!("Time out");
        }
    }
}

```



Barrier

- Le barriere (**Barrier**) sono strumenti di sincronizzazione che consentono a un certo numero di thread di attendersi l'un l'altro a un determinato punto di sincronizzazione.
- In Rust, la libreria standard **std::sync::Barrier** offre una Barrier che può essere utilizzata per questo scopo
- Un thread che arriva al punto di sincronizzazione chiama la funzione **wait()**.

```
use std::sync::{Arc, Barrier};
use std::thread;

fn main() {
    let num_threads = 10;

    // Crea una barriera per sincronizzare num_threads thread
    let barrier = Arc::new(Barrier::new(num_threads));

    let mut handles = Vec::with_capacity(num_threads);

    for i in 0..num_threads {
        let barrier_clone = Arc::clone(&barrier);
        let handle = thread::spawn(move || {
            println!("Thread {} sta facendo un po' di lavoro", i);

            // Simula il lavoro con una sleep crescente per thread
            thread::sleep(std::time::Duration::from_secs((i+1).try_into().unwrap()));

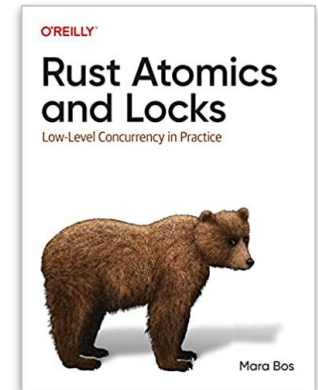
            println!("Thread {} è arrivato alla barriera", i);
            // Attende che tutti i thread raggiungano la barriera
            barrier_clone.wait();

            println!("Thread {} ha superato la barriera", i);
        });
        handles.push(handle);
    }
    for handle in handles { // Attende che tutti i thread completino il loro lavoro
        handle.join().unwrap();
    }
}
```



Per saperne di più

- The C11 and C++11 Concurrency Model, 2014, Mark Batty
 - <https://www.cs.kent.ac.uk/people/staff/mjb211/docs/toc.pdf>
 - Tesi di dottorato in cui viene formalizzato il modello di memoria dei linguaggi C++11/C11
- LLVM Atomic Instructions and Concurrency Guide -
 - <https://llvm.org/docs/Atomics.html>
 - Modello di accesso concorrente alla memoria offerto da LLVM su cui si basa Rust
- The Little Book of Semaphores
 - <https://greenteapress.com/semaphores/LittleBookOfSemaphores.pdf>
- Green threads explained in 200 lines of code
 - <https://cfsamson.gitbook.io/green-threads-explained-in-200-lines-of-rust/>
- Rust Atomics and Locks - Low-Level Concurrency in Practice
 - Mara Bos - O'Reilly 2023 - ISBN: 978-1-098-11944-7
 - Trattazione dettagliata e efficace del modello di concorrenza in Rust



Glossario

- Thread: singola sequenza di istruzioni eseguita all'interno di un processo ed è l'unità base di esecuzione concorrente
- Thread nativo: sono gestiti direttamente dal sistema operativo che è responsabile della sua pianificazione (scheduling), della gestione del suo stato e della sua esecuzione
- Green Thread: meccanismo di concorrenza che emula i thread nativi a livello di spazio utente anziché fare affidamento sul sistema operativo per la gestione, ossia sono thread gestiti da una libreria utilizzata all'interno dell'applicazione, invece che dal kernel del sistema operativo
- Identificatore opaco (handle): identificatore (numero) usato per rappresentare un'entità (file o thread o socket) in modo astratto senza rivelare o esporre dettagli interni; permette al programma di interagire con quella risorsa (gestita dal kernel del sistema operativo) senza accedere direttamente alla sua struttura interna
- Identificatore trasparente (o esplicito): contiene informazioni interpretabili che possono rivelare dettagli sul contesto o sul contenuto (ad es. il path di un file o l'identificatore di un thread)

- Identificatore del thread (TID): identificatore univoco che il sistema operativo assegna a ciascun thread creato all'interno di un processo. Serve per interagire specificamente con quel thread
- Variabile istanza: variabili associate ad una specifica istanza condivisa, ma ogni thread ne possiede una specifica copia
- Istruzioni di barriera alla memoria: istruzioni fornite dalla CPU per imporre un ordine sulle operazioni di memoria eseguite dal processore per garantire la coerenza dei dati in scenari di programmazione concorrente, evitare comportamenti inattesi dovuti al riordino delle operazioni di memoria da parte del processore per ottimizzare le prestazioni e implementare correttamente meccanismi di sincronizzazione di basso livello.

- deadlock (stallo): condizione in cui due o più thread sono bloccati indefinitamente, ciascuno in attesa di una risorsa detenuta da un altro thread
- livelock: condizione in cui due o più thread cambiano continuamente il loro stato in risposta alle azioni di altri thread, senza però compiere alcun progresso verso la loro conclusione: i processi sono attivi, ma improduttivi
- Race condition (corsa critica): comportamento del programma dipende dall'ordine relativo o dalla tempistica di esecuzione di 2 o più thread o processi. Il risultato dell'elaborazione diventa imprevedibile e non deterministico
- Sincronizzazione: insieme di tecniche utilizzate per coordinare l'esecuzione di più thread all'interno di un processo, in modo da gestire correttamente l'accesso a risorse condivise e garantire il un comportamento prevedibile e deterministico del processo.

- Invarianti: condizioni che devono essere sempre vere durante l'esecuzione di un programma
- Invarianti di basso livello: riguardano alcune regole di comportamento per la gestione delle risorse a livello di sistema (ad es. integrità della memoria, allineamento della memoria, gestione dello stack, sincronizzazione nella programmazione concorrente, validità dei puntatori)
- Fearless Concurrency: capacità di scrivere codice concorrente in modo sicuro ed efficiente senza la paura di incorrere nei problemi classici della concorrenza (quali data race e invalidazione della memoria).